Computational Vision

Diego Chiola

Chapter 1

Pixel Basics, Image Types, and Color

Chapter 2

Image Types and Pixel Contents

When we think of digital images, we usually think of standard photographs. However, in computational vision, the value stored inside a pixel depends entirely on what kind of sensor captured the image:

- **Pictorial Digital Images (Photos):** These capture light intensity (yielding black-and-white images) or color.
- **Range Images:** Instead of light, the pixels store **depth information** (how far away an object is from the sensor). This is exactly how tools like Microsoft Kinect or Apple's FaceID operate.
- **Medical Images:** Pixels represent the radiation absorbance level of different tissues (e.g., X-rays or CT scans).
- **Thermal Images:** Pixels capture heat signatures, allowing vision in complete darkness.

Chapter 3

Dynamic Range and Quantization

The **dynamic range** of an image refers to the total number of distinctive values that can possibly occur in the image. Think of this as the "palette" of available shades.

This range is limited by two things: the physical capabilities of the camera sensor and the number of bits (data) we choose to assign to each pixel, a process called quantization.

- **1-bit depth:** The simplest image. A pixel can only be 0 or 1, meaning the image is strictly black and white with no shades of gray.
- **8-bit depth (Gray Level):** This is the standard for grayscale images. We associate 1 byte (which is 8 bits) to each pixel.

3.0.1 Numerical Example:

Because computers run on binary, an 8-bit depth means we have 2^8 possible combinations.

$$2^8 = 256 \text{ gray levels.}$$

This means a pixel can hold any integer value from 0 (pure black) to 255 (pure white). A value of 128 would be a perfect mid-tone gray.

Chapter 4

Color Image Processing

To capture color, cameras use three different physical filters (sensitive to Red, Green, and Blue light) to acquire three separate monochrome images. These three layers are then stacked together to form a single RGB image.

- **Full Color (24-bit):** In standard color images, every single pixel is described by a triplet of values: (R, G, B).
- Because each of the three channels gets its own 8-bit byte (256 possible values), a single color pixel requires 3 bytes of memory.

4.0.1 Numerical Example:

Let's look at the math behind "millions of colors". If Red, Green, and Blue each have 256 possible levels, the total number of color combinations is:

$$256 \times 256 \times 256 = 16,777,216 \text{ possible colors.}$$

For instance, a pixel with the triplet $(255, 0, 0)$ is telling the monitor to display maximum Red, zero Green, and zero Blue. A triplet of $(255, 255, 255)$ mixes all light to create pure white.

When processing these images algorithmically, engineers typically face a choice: either process the R, G, and B channels completely independently and merge the results at the end, or devise ad-hoc algorithms that analyze the complex interplay of the colors together.

Chapter 5

Global Descriptors and Image Segmentation

5.1 The Goal: Image Segmentation

One of the most fundamental tasks in computational vision is **image segmentation**. This means dividing an image into meaningful parts, essentially detecting groups of pixels that are physically close together and visually similar.

We can evaluate "similarity" based on various attributes, such as:

- **Brightness** (in grayscale images)
- **Color**
- **Texture** or **Motion**

The simplest example is separating a foreground object (like the cameraman in the slides) from the background.

5.2 Brightness Histograms and Binarization

To figure out which pixels belong to the foreground and which belong to the background, we use a **brightness histogram**.

- **What is it?** A histogram is a graph where the x-axis represents the possible pixel values (0 to 255 for an 8-bit image) and the y-axis represents the number of pixels in the image that have that exact value.

If an image has a dark background and a bright subject, the histogram will typically show two distinct "peaks" (a bimodal histogram). We can achieve segmentation through **binarization**. This involves picking a threshold value directly between those two peaks.

Numerical Example:

Imagine an image where the background pixels mostly have values around 30 (dark gray) and the object pixels are around 200 (light gray). We can set a threshold at $T = 100$. The algorithm looks at every pixel $I(x, y)$:

- If $I(x, y) < 100$, it sets the pixel to 0 (pure black).
- If $I(x, y) \geq 100$, it sets the pixel to 255 (pure white).

The result is a clean binary image (a silhouette).

5.3 The Challenge of Contrast

The slides point out a critical reality: finding that perfect threshold is rarely easy! The shape of the histogram tells us a lot about the image quality:

- **Dark image:** All pixels are bunched up on the left side (near 0).
- **Bright image:** All pixels are bunched up on the right side (near 255).
- **Low-contrast image:** The pixels are squished together in a narrow spike in the middle. It's very hard to separate objects here.
- **High-contrast image:** The pixels are spread widely across the entire 0-255 range. This is ideal for thresholding.

5.4 Dealing with Color

When we move to color images, segmentation gets trickier. The slides note that in color, values being numerically "close" in raw RGB code doesn't always mean they look visually "similar" to human eyes.

For example, a pixel in shadow might have entirely different RGB numbers than a sunlit pixel of the exact same object. Because of this, a careful choice of the **color space** (like moving from RGB to HSV - Hue, Saturation, Value) is highly recommended before trying to segment.

5.5 Image Quantization

Another technique mentioned is **Image Quantization**, which is the deliberate reduction of the image's dynamic range. Instead of having 256 shades of gray, we might force the image to only use 4 or 8 shades. This simplifies the image drastically, washing out minor noise and making it a very useful pre-processing step before attempting segmentation.

Numerical Example:

Let's say we want to reduce 256 levels down to just 4 levels. We divide the range into 4 "buckets" of 64 values each. For a pixel with an original value of 150:

$$New\_Value = \left\lfloor \frac{150}{64} \right\rfloor = 2$$

We map it to bucket "2", completely erasing the subtle differences between value 150 and value 160 (they both go into the same bucket).

5.6 Summary of Global Descriptors (Histograms)

Histograms are used to obtain an overall description of the image content.

- **The Good:** They are very easy to compute and highly robust to geometrical variations. If you take a picture of an apple and rotate the photo 90 degrees, the histogram remains exactly the same.
- **The Bad:** They completely **lose spatial information**.

Context: To understand why losing spatial information is bad, imagine an image of a chessboard. Now imagine a second image where the top half is solid black and the bottom half is solid white. Both images have 50% black pixels and 50% white pixels. Their histograms will be absolutely identical, even though the images look completely different!

Chapter 6

Linear Filters and Local Features

6.1 Linear Filtering and Convolution

A basic tool in image processing is the linear filter, used for noise reduction and signal enhancement. Filtering is achieved through a mathematical operation called **convolution**.

In convolution, a small matrix called a **kernel** (k) is swept across the image signal (f) to produce a new filtered image (g):

$$g = f * k$$

For a 2D image, the math is the sum of the element-wise products between the kernel and the image patch it currently covers:

$$g[x, y] = \sum_{m,l} f[x - m, y - l]k[m, l]$$

Numerical Example:

Imagine a 1D row of pixels transitioning from dark to light: $f = [10, 10, 10, 50, 50, 50]$. We want to find the edge using a simple edge-detection kernel: $k = [-1, 0, 1]$. Let's center the kernel over the third pixel (value 10):

$$g[3] = (10 \times -1) + (10 \times 0) + (50 \times 1) = -10 + 0 + 50 = 40$$

The high output value (40) indicates that a sharp transition (an edge) was detected at this location.

6.2 Image Gradients and Edges

Edges are pixels at which the image values undergo a sharp variation. To find them, we compute the **Image Gradient**, a vector that points in the direction of the most rapid change in intensity.

The gradient is composed of partial derivatives in the x and y directions:

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

From this, we extract two crucial pieces of geometric information:

- **Gradient Magnitude** (Edge strength): $M(x, y) = \sqrt{g_x^2 + g_y^2}$
- **Gradient Orientation** (Edge direction): $\theta(x, y) = \arctan\left(\frac{g_y}{g_x}\right)$

6.3 Image Gradient: Numerical Example

To calculate the gradient at a specific pixel, we estimate the partial derivatives using the **central difference** of its immediate neighbors.

Imagine a 3×3 patch of grayscale image pixels, where we want to find the gradient of the center pixel (x, y):

$$\text{Patch} = \begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{50} & 90 \\ 10 & 90 & 170 \end{bmatrix}$$

6.3.1 Calculate Partial Derivatives

We find the rate of change in the horizontal (x) and vertical (y) directions. *(Note: In standard image coordinates, the y-axis increases downwards).*

- **Horizontal change (g_x):** Pixel to the Right - Pixel to the Left

$$g_x = f(x+1, y) - f(x-1, y) = 90 - 10 = 80$$

- **Vertical change (g_y):** Pixel Below - Pixel Above

$$g_y = f(x, y+1) - f(x, y-1) = 90 - 10 = 80$$

6.3.2 Calculate Gradient Magnitude

The magnitude measures the **strength** of the edge.

$$M(x, y) = \sqrt{g_x^2 + g_y^2}$$

$$M = \sqrt{80^2 + 80^2} = \sqrt{6400 + 6400} = \sqrt{12800} \approx 113.1$$

6.3.3 Calculate Gradient Orientation

The orientation measures the **direction** of the most rapid increase in intensity.

$$\theta(x, y) = \arctan\left(\frac{g_y}{g_x}\right)$$

$$\theta = \arctan\left(\frac{80}{80}\right) = \arctan(1) = 45^\circ$$

Conclusion: The gradient vector points at a 45° angle (down and to the right). The gradient always points in the direction of steepest ascent (from dark to bright).

6.4 The Effect of Noise and Gaussian Smoothing

Calculating derivatives is highly sensitive to noise. Taking the derivative of a noisy signal results in massive spikes that hide the true edges.

The Solution: We must smooth the image before taking the derivative. We typically use a **Gaussian Filter**, which acts as a low-pass filter to blur out random noise.

$$h_\sigma(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

Mathematical Shortcut: Because convolution is associative, taking the derivative of a smoothed image is mathematically identical to smoothing the image with the *derivative of the smoothing filter*:

$$\frac{\partial}{\partial x}(h * f) = \left(\frac{\partial}{\partial x} h\right) * f$$

This saves computational power by allowing us to use a single "Derivative of Gaussian" filter. This logic forms the foundation of the famous **Canny Edge Detector**.

6.5 Local Features: Corners

While edges are useful, they suffer from the aperture problem (you cannot tell where along an edge you are). **Corners** are patches with strong gradients in at least *two* significantly different orientations, making them stable and "good features to match" across different images.

6.5.1 The Autocorrelation Function

To detect corners, we analyze how a local patch of pixels changes when shifted by a small vector u. We use the Summed Square Difference (SSD), also known as the autocorrelation function $E_{AC}(u)$:

$$E_{AC}(u) = \sum_i [I(x_i + u) - I(x_i)]^2$$

To make this computationally feasible, we approximate the shifted image using a Taylor series expansion:

$$I(x_i + u) \approx I(x_i) + \nabla I(x_i) \cdot u$$

Substituting this back into the SSD equation cancels out the $I(x_i)$ terms, yielding a quadratic form:

$$E_{AC}(u) \approx \sum_i [\nabla I(x_i) \cdot u]^2 = u^\top A u$$

6.5.2 The Autocorrelation Matrix (A)

The matrix A captures the gradient distribution within the patch:

$$A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

By calculating the eigenvalues (λ_1, λ_2) of A, we can classify the local geometry:

- **Corner:** λ_1 and λ_2 are both large.
- **Edge:** $\lambda_1 \gg \lambda_2$ or $\lambda_2 \gg \lambda_1$.
- **Flat Region:** Both λ_1 and λ_2 are small.

6.5.3 Corner Numerical Example

Consider a 4×4 image grid containing a dark square (intensity 10) on a light background (intensity 100). We will compute the Autocorrelation Matrix for the inner 2×2 patch.

$$\text{Image} = \begin{bmatrix} 100 & 100 & 100 & 100 \\ 100 & \mathbf{10} & \mathbf{10} & 100 \\ 100 & \mathbf{10} & \mathbf{10} & 100 \\ 100 & 100 & 100 & 100 \end{bmatrix}$$

Step 1: Calculate Central Difference Gradients (I_x, I_y) For the four inner pixels:

- **Top-Left:** $I_x = 10 - 100 = -90$, $I_y = 10 - 100 = -90$
- **Top-Right:** $I_x = 100 - 10 = 90$, $I_y = 10 - 100 = -90$
- **Bottom-Left:** $I_x = 10 - 100 = -90$, $I_y = 100 - 10 = 90$
- **Bottom-Right:** $I_x = 100 - 10 = 90$, $I_y = 100 - 10 = 90$

Step 2: Calculate Matrix Components Squaring and summing these values over the 4 pixels:

$$\sum I_x^2 = (-90)^2 + (90)^2 + (-90)^2 + (90)^2 = 32400$$

$$\sum I_y^2 = (-90)^2 + (-90)^2 + (90)^2 + (90)^2 = 32400$$

$$\sum I_x I_y = 8100 - 8100 - 8100 + 8100 = 0$$

Step 3: Construct Matrix A and Extract Eigenvalues

$$A = \begin{bmatrix} 32400 & 0 \\ 0 & 32400 \end{bmatrix}$$

Because A is a diagonal matrix, its eigenvalues are simply the diagonal entries:

$$\lambda_1 = 32400$$

$$\lambda_2 = 32400$$

Conclusion: Because both eigenvalues are exceedingly large, the algorithm confidently classifies this patch as a **Corner**.

Chapter 7

Image Segmentation

Image segmentation is a fundamental task in computational vision that involves partitioning a digital image into multiple segments (sets of pixels, also known as image objects). The goal of segmentation is to simplify and/or change the representation of an image into something that is more meaningful and easier to analyze.

7.1 Core Concept: Uniformity

Segmentation typically divides an image into regions that are uniform according to a specific quality or feature:

- **Brightness/Intensity:** Common light levels.
- **Color:** Similarities in the color space.
- **Motion:** Pixels moving with the same velocity/direction.

7.2 Mathematical Definition

Let R represent the entire spatial region occupied by an image. Image segmentation is the process of partitioning R into n subregions $R_1, R_2, \ldots, R_n$ such that:

1. **Completeness:** $\bigcup_{i=1}^{n} R_i = R$. Every pixel must belong to a region.
2. **Connectivity:** R_i is a connected set for $i = 1, \ldots, n$.
3. **Disjointness:** $R_i \cap R_j = \emptyset$ for all $i \neq j$. Regions cannot overlap.
4. **Uniformity Predicate:** $Q(R_i) = \text{TRUE}$ for $i = 1, \ldots, n$. All pixels in a region satisfy a logical predicate Q (e.g., same intensity).
5. **Maximality:** $Q(R_i \cup R_j) = \text{FALSE}$ for any adjacent regions R_i and R_j. Neighboring regions must be different according to the predicate Q.

7.3 Methods Overview

Segmentation methods can be broadly categorized based on the level of guidance and the visual cues used.

7.3.1 Supervision Levels

- **Unsupervised:** Segmentation is based purely on pixel and appearance information (low-level features).
- **Supervised:** Segmentation is guided by semantic concepts or prior knowledge of specific object classes.

7.3.2 Technical Approaches

- **Edge-based methods:** Look for discontinuities (edges) in the image to find boundaries between regions.
- **Region-based methods:** Look for regions of similarity according to specific criteria. This is generally considered a more robust formulation.

Chapter 8

Thresholding and Motion Segmentation

This chapter focuses on **Thresholding**, a region-based segmentation method that groups pixels sharing similar intensity levels.

8.1 Basic Thresholding Concepts

The simplest form of segmentation is the binary case, where we separate an image into two classes: Foreground and Background. This is achieved by comparing each pixel intensity $f(x, y)$ to a global threshold T.

8.1.1 Global Binary Thresholding

The resulting image $g(x, y)$ is defined as:

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) > T \\ 0 & \text{otherwise} \end{cases} \tag{8.1}$$

Where 1 typically represents the foreground and 0 the background.

8.1.2 Multiple Thresholding

When an image contains multiple objects with distinct intensity ranges, a single threshold is insufficient. Multiple thresholds $(T_1, T_2, \dots)$ can be used:

$$g(x,y) = \begin{cases} a & \text{if } f(x,y) > T_2 \\ b & \text{if } T_1 < f(x,y) \leq T_2 \\ c & \text{otherwise} \end{cases} \tag{8.2}$$

8.2 Motion Segmentation

Motion segmentation is a practical application of thresholding where the "uniformity" criterion is motion rather than static intensity. By identifying pixels that change significantly between frames, we can extract moving objects from a static background. This is a binary segmentation task (Moving vs. Not Moving).

8.3 Automatic Global Threshold Selection

Instead of manually picking T, we can use an iterative algorithm to find the optimal value, typically located in the "valley" of a bimodal histogram.

8.3.1 The Iterative Algorithm

1. **Initial Estimate:** Select an initial T (often the global mean intensity of the image).
2. **Segmentation:** Partition the image into two groups: G_1 (pixels $> T$) and G_2 (pixels $\leq T$).
3. **Mean Computation:** Calculate the average intensity values m_1 and m_2 for groups G_1 and G_2 respectively.
4. **Update:** Compute a new threshold $T = \frac{1}{2}(m_1 + m_2)$.
5. **Iteration:** Repeat steps 2 through 4 until T stabilizes (the change is smaller than a predefined epsilon).

8.3.2 Numerical Example

Suppose we have a set of 16 pixel intensities:

$$\{10, 12, 15, 20, 22, 25, 100, 110, 120, 130, 140, 150, 160, 170, 180, 200\}$$

Iteration 1:

- **Initial T_0:** Global mean ≈ 101.
- **Groups:** $G_1 = \{110, \ldots, 200\}$ (9 pixels), $G_2 = \{10, \ldots, 100\}$ (7 pixels).
- **Means:** $m_1 = 151.1$, $m_2 = 29.1$.
- **New T:** $\frac{1}{2}(151.1 + 29.1) = 90.1$.

Iteration 2:

- **Thresholding with $T = 90.1$:** The value 100 now moves from G_2 to G_1.
- **New Groups:** $G_1 = \{100, 110, \ldots, 200\}$ (10 pixels), $G_2 = \{10, \ldots, 25\}$ (6 pixels).
- **New Means:** $m_1 = 146$, $m_2 = 17.3$.
- **New T:** $\frac{1}{2}(146 + 17.3) = 81.65$.

The algorithm continues until the threshold value converges, effectively separating the dark cluster from the light cluster.

Chapter 9

Otsu's Method for Global Thresholding

While iterative methods rely on heuristic averaging, Otsu's method is grounded in statistical discriminant analysis. Its objective is to segment foreground and background by maximizing the **between-class variance**. Computations are performed on the normalized image histogram.

9.1 Mathematical Formulation

Let the image have intensity levels from 0 to $L-1$. The normalized histogram gives the probability p_i of each intensity level i.

We define a threshold k ($0 < k < L-1$) that divides pixels into two classes:

- C_1: Pixels with intensities $[0, k]$
- C_2: Pixels with intensities $[k+1, L-1]$

The probabilities of a pixel belonging to C_1 or C_2 are:

$$P_1(k) = \sum_{i=0}^{k} p_i \tag{9.1}$$

$$P_2(k) = \sum_{i=k+1}^{L-1} p_i = 1 - P_1(k) \tag{9.2}$$

The mean intensities of the classes are:

$$m_1(k) = \frac{1}{P_1(k)} \sum_{i=0}^{k} i p_i \tag{9.3}$$

$$m_2(k) = \frac{1}{P_2(k)} \sum_{i=k+1}^{L-1} i p_i \tag{9.4}$$

The global mean of the entire image is $m_G = \sum_{i=0}^{L-1} i p_i$.

9.2 Optimization Criterion

Otsu's method finds the optimal threshold k^* that maximizes the between-class variance $\sigma_B^2(k)$:

$$k^* = \arg \max_{0 \le k \le L-1} \sigma_B^2(k) \tag{9.5}$$

Where the between-class variance is defined as:

$$\sigma_B^2(k) = P_1(k)(m_1(k) - m_G)^2 + P_2(k)(m_2(k) - m_G)^2 \tag{9.6}$$

9.3 Numerical Example

Consider a 10-pixel image with 4 intensity levels ($L = 4$). Pixel values: $\{0, 0, 0, 1, 1, 2, 3, 3, 3, 3\}$.

Probabilities: $p_0 = 0.3$, $p_1 = 0.2$, $p_2 = 0.1$, $p_3 = 0.4$. Global Mean: $m_G = 0(0.3) + 1(0.2) + 2(0.1) + 3(0.4) = 1.6$.

We evaluate the variance for potential thresholds k:

- **For $k = 0$:** $P_1 = 0.3, P_2 = 0.7, m_1 = 0, m_2 \approx 2.28$.
  $\sigma_B^2(0) = 0.3(0 - 1.6)^2 + 0.7(2.28 - 1.6)^2 \approx 1.09$.
- **For $k = 1$:** $P_1 = 0.5, P_2 = 0.5, m_1 = 0.4, m_2 = 2.8$.
  $\sigma_B^2(1) = 0.5(0.4 - 1.6)^2 + 0.5(2.8 - 1.6)^2 = 1.44$.
- **For $k = 2$:** $P_1 = 0.6, P_2 = 0.4, m_1 \approx 0.67, m_2 = 3$.
  $\sigma_B^2(2) = 0.6(0.67 - 1.6)^2 + 0.4(3 - 1.6)^2 \approx 1.30$.

The maximum variance is 1.44 at $k = 1$. Therefore, the optimal threshold is $T = 1$.

Chapter 10

Improving Thresholding

Global thresholding can fail if the image is noisy or if lighting is uneven.

10.1 Image Smoothing

Noisy images produce jagged histograms where it is difficult to find a clear valley. Applying Gaussian smoothing prior to thresholding smooths the histogram, making global thresholding algorithms (like Otsu's) significantly more effective.

10.2 Variable (Local) Thresholding

When an image has uneven illumination (e.g., shadows across a document), a single global threshold will fail. Variable thresholding solves this by subdividing the image or computing a unique threshold T_{xy} for every pixel based on its local neighborhood.

A common approach computes the threshold based on local mean (m_{xy}) and standard deviation (σ_{xy}):

$$T_{xy} = a\sigma_{xy} + bm_{xy} \quad (a, b \geq 0) \tag{10.1}$$

For text segmentation, a **moving average** approach is highly effective. It calculates the threshold $T_{xy} = bm_{xy}$ where m is estimated using only the last n visited pixels along a scan line.

Chapter 11

Color Segmentation and K-Means Clustering

When moving from grayscale to color images, the feature space expands to multiple dimensions (e.g., 3D spaces like RGB or HSV). For generic color segmentation where no prior target color is specified, clustering methods like K-means are highly effective.

11.1 K-Means Clustering: The Concept

K-means is an unsupervised clustering algorithm. In the context of image segmentation, it plots every pixel as a point in a multidimensional color space and attempts to group these points into K distinct clusters based on their proximity (color similarity).

11.2 The Mathematical Objective

Given a set of image pixels $(x_1, \ldots, x_n)$, the goal is to partition the data into K sets, $S = (S_1, \ldots, S_K)$, to minimize the within-cluster variance. The objective function is:

$$\arg\min_{S} \sum_{i=1}^{K} \sum_{x \in S_i} ||x - \mu_i||^2 \tag{11.1}$$

Where:

- K is the predefined number of clusters.
- x is the feature vector of a pixel (e.g., its RGB values).
- μ_i is the centroid (mean vector) of cluster S_i.

11.3 The Iterative Algorithm

K-means minimizes the objective function through the following iterative steps:

1. **Random Initialization:** Select K random points in the feature space to serve as the initial centroids.
2. **Points Association:** Calculate the distance between every pixel x and all K centroids. Assign each pixel to the cluster with the nearest centroid.
3. **Centroids Update:** Recalculate the position of the K centroids by taking the mean of all pixels currently assigned to each cluster.
4. **Convergence Check:** Repeat Steps 2 and 3 until stability is reached (i.e., pixel assignments no longer change or centroid movement falls below a threshold).

11.4 Numerical Example: RGB Clustering

Consider an image with 6 pixels and $K = 2$ clusters.

1. **Data Points (RGB):**

- $P_1(200, 0, 0), P_2(180, 20, 20), P_3(220, 10, 10)$ (Reddish)
- $P_4(0, 150, 0), P_5(10, 170, 10), P_6(20, 130, 20)$ (Greenish)

2. **Initialization:** Assume initial centroids $C_1 = P_1(200, 0, 0)$ and $C_2 = P_4(0, 150, 0)$.
3. **Assignment Step (Iteration 1):** Using Euclidean distance $D = \sqrt{\Delta R^2 + \Delta G^2 + \Delta B^2}$:

- For P_2: $Dist(P_2, C_1) \approx 34.6$ vs $Dist(P_2, C_2) \approx 222.9 \rightarrow$ Cluster 1.
- For P_5: $Dist(P_5, C_1) \approx 255.1$ vs $Dist(P_5, C_2) \approx 24.5 \rightarrow$ Cluster 2.

After checking all points, Cluster 1 contains $\{P_1, P_2, P_3\}$ and Cluster 2 contains $\{P_4, P_5, P_6\}$.

4. **Update Step:** Calculate new centroids by taking the mean of assigned pixels:

- $C_{1\_new} = \left(\frac{200+180+220}{3}, \frac{0+20+10}{3}, \frac{0+20+10}{3}\right) = (200, 10, 10)$
- $C_{2\_new} = \left(\frac{0+10+20}{3}, \frac{150+170+130}{3}, \frac{0+10+20}{3}\right) = (10, 150, 10)$

The centroids have moved closer to the center of their respective color clouds. The process repeats until assignments no longer change.

11.5 Extended Feature Space: RGBXY

Standard K-means on RGB values groups pixels strictly by color, ignoring their physical location in the image. A red pixel in the top-left corner and a red pixel in the bottom-right corner will be placed in the same cluster. To encourage spatially contiguous segments, the feature space can be expanded to 5 dimensions: **RGBXY**. This forces the algorithm to consider both color similarity and spatial proximity when calculating the distance.

11.6 Pros, Cons, and the Choice of K

- **Pros:** The algorithm is simple to implement, understand, and computationally relatively fast.
- **Cons:** Results can be non-deterministic due to the random initialization of centroids. Furthermore, it does not inherently prevent the formation of highly unbalanced clusters.
- **Choosing K:** The number of clusters K must be set a priori. A common solution to finding the optimal K is to run the algorithm across multiple values of K (e.g., $K = 2, 3, 4, 5$) and evaluate the results using a cluster quality metric, such as the **Davies-Bouldin index**, to find the partition that yields the most distinct and cohesive clusters.

Chapter 12

Superpixels and the SLIC Algorithm

Superpixels provide a convenient primitive from which to compute local image features. They group pixels into compact, roughly uniform regions that respect object boundaries, capturing redundancy in the image and drastically reducing the complexity of subsequent computer vision tasks.

12.1 SLIC: Simple Linear Iterative Clustering

SLIC is an adaptation of the K-means algorithm designed specifically for generating superpixels. It performs a local clustering of pixels in a 5-dimensional space defined by $[l, a, b, x, y]$.

- l, a, b: The pixel color vector in the CIELAB color space, chosen for its perceptual uniformity.
- x, y: The spatial position of the pixel in the image.

12.2 The Distance Metric (D_s)

Because CIELAB color distances and spatial plane distances have different scales, they cannot be directly combined using a simple Euclidean norm. SLIC introduces a specialized distance measure D_s that normalizes the spatial distance.

Given a cluster center $C_k = [l_k, a_k, b_k, x_k, y_k]^T$ and a pixel i at $[l_i, a_i, b_i, x_i, y_i]^T$, the distances are computed as follows:

$$d_{lab} = \sqrt{(l_k - l_i)^2 + (a_k - a_i)^2 + (b_k - b_i)^2} \tag{12.1}$$

$$d_{xy} = \sqrt{(x_k - x_i)^2 + (y_k - y_i)^2} \tag{12.2}$$

$$D_s = d_{lab} + \frac{m}{S} d_{xy} \tag{12.3}$$

Parameters:

- K: The desired number of superpixels (the only required user input).
- S: The regular grid interval between superpixels, defined as $S = \sqrt{N/K}$ where N is the total number of pixels.
- m: The compactness parameter. A higher m emphasizes spatial proximity, resulting in more compact, rigidly square superpixels.

12.3 Algorithm Complexity ($O(N)$)

Standard K-means evaluates distances from every pixel to every cluster center, resulting in $O(NKI)$ complexity. SLIC drastically reduces this by localizing the search. Since a superpixel occupies roughly S^2 area, SLIC restricts the pixel search to a $2S \times 2S$ region surrounding the cluster center. Consequently, a pixel is evaluated against no more than eight centers, granting the algorithm a strictly linear $O(N)$ complexity, regardless of K.

12.4 Algorithm Steps

1. **Initialization:** Sample K cluster centers at regular grid intervals S. Initialize distance $d(i) = \infty$ and label $l(i) = -1$ for all pixels.
2. **Perturbation:** Move centers to the lowest gradient position in a 3×3 neighborhood to avoid noisy edges.
3. **Assignment:** For each center k, evaluate pixels in a $2S \times 2S$ region. Compute D_s. If $D_s < d(i)$, update $d(i) = D_s$ and assign the label $l(i) = k$.
4. **Update:** Recompute centers as the mean $[l, a, b, x, y]$ vector of all associated pixels.
5. **Convergence:** Repeat assignment and update until residual error converges.
6. **Connectivity:** Enforce spatial connectivity by merging small, stray, or disjoint segments with the largest neighboring cluster.

12.5 Numerical Example: Label Assignment

Suppose an image has $N = 100$ pixels and we want $K = 4$ superpixels.

- Grid interval $S = \sqrt{100/4} = 5$.
- Compactness parameter $m = 10$.

Target Pixel P_i: Located at $(4, 5)$ with color $(60, 0, 0)$. Initial state: $l(i) = -1$, $d(i) = \infty$.
Testing Cluster Center 1 (C_1): Located at $(2, 2)$ with color $(50, 0, 0)$.

$$\begin{aligned} d_{lab} &= \sqrt{(50-60)^2} = 10 \\ d_{xy} &= \sqrt{(2-4)^2 + (2-5)^2} = \sqrt{13} \approx 3.61 \\ D_s &= 10 + \left(\frac{10}{5}\right) \times 3.61 = 17.22 \end{aligned}$$

Since $17.22 < \infty$, update pixel: $d(i) = 17.22$, $l(i) = 1$.
Testing Cluster Center 2 (C_2): Located at $(7, 5)$ with color $(65, 0, 0)$.

$$\begin{aligned} d_{lab} &= \sqrt{(65-60)^2} = 5 \\ d_{xy} &= \sqrt{(7-4)^2 + (5-5)^2} = \sqrt{9} = 3 \\ D_s &= 5 + \left(\frac{10}{5}\right) \times 3 = 11 \end{aligned}$$

Since $11 < 17.22$, update pixel: $d(i) = 11$, $l(i) = 2$.
Pixel P_i successfully finds its nearest center and is assigned to Cluster 2.

Chapter 13

The Goal of Image Matching

The core problem introduced in this section is determining how similar two images are, which is highly dependent on the application. The course breaks this down into two main approaches:

- **Global Matching:** This involves computing a single, overarching descriptor for the entire image, such as a color histogram or overall brightness mean. This is typically used for general tasks like image retrieval or broad image classification.
- **Local Matching:** Instead of looking at the whole image, this method focuses on finding specific, localized "points of interest" and matching them across different images. This is crucial for precise tasks like 3D reconstruction and image registration.

13.1 The Local Matching Pipeline

We focuses heavily on the Local Matching approach. The standard three-step pipeline forms the backbone of the rest of the course:

1. **Feature Detection:** Finding the specific "interest points" (like corners or blobs) on each image.
2. **Feature Description:** Computing a mathematical vector that describes the visual neighborhood around each detected point.
3. **Feature Matching:** Comparing these descriptions between two images to find the matching pairs.

13.2 Degrees of Difficulty in Matching

Local matching ranges from "easy" to "much harder" based on the transformations between the two images:

- **Easy:** The images only have simple translations (shifting up/down/left/right).
- **Harder:** The images undergo translation, changes in illumination, and slight deformations.
- **Much Harder:** The images feature dramatic changes in the 3D viewpoint and heavy illumination changes.

13.3 Integration from the DoG and SIFT Paper

To solve these "much harder" cases, like the NASA Mars Rover images, we cannot rely on simple pixel comparisons. David Lowe's paper introduces **SIFT (Scale-Invariant Feature Transform)** specifically to address this. The goal is to transform image data into coordinates that are invariant to image scaling and rotation, and highly robust against 3D viewpoint changes and lighting variations. This allows a single local feature to find its correct match even in a massive database of features.

Chapter 14

Scale Invariant Feature detectors

In Chapter 1, we established the need to find "interest points" (like corners or blobs). However, simple corner detectors fail when the distance to the object changes. A leaf might look like a tiny blob from 100 meters away, but show sharp corners from 1 meter away. To perform robust matching, we need an algorithm that finds features regardless of how zoomed in or out the image is. David Lowe's SIFT (Scale-Invariant Feature Transform) solves this.

14.1 Building "Scale-Space"

To teach a computer to look at an image at different scales, we blur it. Blurring removes high-frequency details, simulating how an object appears from further away.

The "scale space" of an image is a function $L(x, y, \sigma)$, produced by convolving the original image $I(x, y)$ with a 2D Gaussian blur filter $G(x, y, \sigma)$:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \tag{14.1}$$

Where σ (sigma) is the scale (amount of blur).

14.2 The Blob Finder: Laplacian of Gaussian (LoG)

To find interesting features at these scales, we look for "blobs" (uniform areas surrounded by sharp variations). The mathematical tool for finding blobs is the Laplacian, which measures the 2nd spatial derivative of the image.

$$\nabla^2 L = \frac{\partial^2 L}{\partial x^2} + \frac{\partial^2 L}{\partial y^2} \tag{14.2}$$

14.2.1 Numerical Example: 1D LoG

Imagine a 1D row of pixel intensities with a bright blob in the center:

$$I = [0, 0, 10, 10, 0, 0]$$

The 1st derivative (difference between adjacent pixels) highlights the edges:

$$I' = [0, 10, 0, -10, 0]$$

The 2nd derivative (difference of differences) highlights the center of the peak and the surrounding dip:

$$I'' = [10, -10, -10, 10]$$

When applied in 2D to a blurred image, this filter looks like a "Mexican hat". The algorithm searches for the specific σ where this 2nd derivative response is absolutely maximized.

14.3 The Genius Shortcut: Difference of Gaussians (DoG)

Calculating the 2nd derivative for every pixel at every possible scale is computationally expensive. Lowe's SIFT provides an optimization based on the heat diffusion equation: subtracting two images with different levels of Gaussian blur closely approximates the normalized Laplacian of Gaussian.

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \tag{14.3}$$

14.4 How Different Should the Two Scales Be? (The multiplier k)

This is a crucial parameter in SIFT. We don't just pick random blur levels. The scale space is divided into "octaves". An octave corresponds to doubling the value of σ. Within each octave, Lowe divides the space into an integer number of intervals, s. The scale multiplier k between two adjacent blurred images is defined as:

$$k = 2^{1/s} \tag{14.4}$$

In his foundational paper, Lowe recommends $s = 3$ intervals per octave. Therefore, the difference in scale between two adjacent blurred images is $k = 2^{1/3} \approx 1.2599$. If your base scale is $\sigma = 1.6$, the next image in the stack is blurred at $\sigma = 1.6 \times 1.26 = 2.016$.

14.4.1 Numerical Example: DoG Calculation

Suppose at pixel coordinate $(50, 50)$, the intensity in the first blurred image $L(x, y, \sigma)$ is 145. In the next progressively blurred image $L(x, y, k\sigma)$, the intensity is 130. The DoG value for that pixel at that scale is:

$$D(50, 50, \sigma) = 130 - 145 = -15 \tag{14.5}$$

14.5 Extrema Detection (Finding the Keypoints)

Once we have a 3D stack of DoG images (X, Y, and Scale), we look for local extrema (maxima or minima) to identify keypoints.

A single pixel is compared against its 26 neighbors in the 3D scale-space:

- 8 adjacent pixels in its current scale ($k\sigma$).
- 9 pixels directly above it in the next scale ($k^2\sigma$).
- 9 pixels directly below it in the previous scale (σ).

14.6 Numerical Example: 3D Neighborhood Check

Imagine a $3 \times 3 \times 3$ cube of DoG values. The center pixel is our candidate.

- **Scale σ (Below):** All 9 pixels have values ranging from -5 to 12.
- **Scale $k\sigma$ (Current):** The center candidate has a value of **45**. Its 8 immediate neighbors range from 20 to 38.
- **Scale $k^2\sigma$ (Above):** All 9 pixels have values ranging from 15 to 30.

Because 45 is strictly greater than all 26 surrounding values, this coordinate $(x, y, k\sigma)$ is officially recorded as a keypoint!

14.7 Filtering Bad Keypoints (Additional Info)

Simply finding extrema isn't enough; some points are unstable. SIFT applies two more filters:

1. **Low Contrast Filter:** If the absolute DoG value at the extremum is less than a threshold (Lowe uses 0.03), it is discarded as noise.
2. **Edge Response Elimination:** DoG has a strong response along flat edges, which are terrible for matching (the aperture problem). SIFT computes the Hessian matrix (2nd derivatives in x and y) at the keypoint. If the ratio of the principal curvatures indicates it's an edge rather than a blob, the point is discarded.

Chapter 15

Feature descriptors

In Chapter 2, we found the (x, y) coordinates and scale (σ) of our keypoints. To match them, we need to describe what the visual area around them looks like.

15.1 The Problem with "Raw Patches"

The simplest method is to take a square window (a patch) of pixels around the keypoint and list the pixel intensities. However, if the camera rotates even slightly between two images, the pixels in that square window will completely shift. A pixel-by-pixel comparison (Sum of Squared Differences) will fail. We need a descriptor that is **invariant to rotation**.

15.2 Clarifying the Keypoint Neighborhood

A common confusion is assuming the descriptor is built from a single pixel. A keypoint is just a central coordinate. SIFT builds its descriptor by analyzing a **neighborhood** of pixels around that center point. The size of this neighborhood depends on the keypoint's characteristic scale (σ) found in Chapter 2.

For every pixel in this neighborhood, the computer calculates two values using finite differences:

- **Gradient Magnitude** (m)**:** How sharp the contrast is (edge strength).
- **Gradient Orientation** (θ)**:** Which direction is the brightest (from 0° to 360°).

15.3 Finding "North": The 36-Bin Orientation Histogram

To make our descriptor immune to rotation, SIFT gives each keypoint its own distinct "North" based on the visual data around it.

15.3.1 How the 36 Bins Work

The algorithm creates a histogram with 36 bins. Since a full circle is 360°, each bin covers exactly 10°.

- Bin 1: 0° to 9°
- Bin 2: 10° to 19°
- ...
- Bin 36: 350° to 359°

15.3.2 Numerical Example: Voting for an Orientation

Imagine the neighborhood around our keypoint has 50 pixels. We calculate the angle and magnitude for each:

- **Pixel A:** $\theta = 12^\circ$, Magnitude $= 5$. It belongs to **Bin 2**. We add 5 to Bin 2.

- **Pixel B:** $\theta = 18°$, Magnitude = 10. It also belongs to **Bin 2**. We add 10. (Bin 2 is now 15).
- **Pixel C:** $\theta = 95°$, Magnitude = 2. It belongs to **Bin 10**. We add 2 to Bin 10.

After processing all 50 pixels, we look at the 36 bins. Suppose **Bin 5 ($40° - 49°$)** has the highest total magnitude (a massive spike in the histogram). This means the dominant visual feature around this keypoint is pointing at roughly 45°. SIFT now rotates the entire coordinate system of this patch by 45°. **The patch is now rotationally invariant!**

15.4 Building the 128-Dimensional SIFT Descriptor

Now that the patch is scaled and rotated correctly, we build the actual mathematical fingerprint.

15.4.1 The Grid and the 8 Bins

1. Take a 16×16 **window** of pixels centered on the keypoint.
2. Divide this window into a 4×4 **grid**. This gives us 16 distinct "sub-regions", each containing $4 \times 4 = 16$ pixels.
3. Inside each of the 16 sub-regions, we create a new, smaller histogram. This one only has **8 bins** (each covering 45°: N, NE, E, SE, S, SW, W, NW).
4. We tally up the gradients of the 16 pixels into these 8 bins.

15.4.2 What Information is in the 8 Bins?

The 8 bins describe the geometry of the lines inside that specific sub-region. For example, look at the top-left sub-region. If it contains a sharp horizontal edge, the gradients will point straight up or straight down. Therefore, the "North" (90°) and "South" (270°) bins will have huge numbers, while the other 6 bins will be close to zero.

15.4.3 The Final Vector

You have 16 sub-regions. Every sub-region gives you 8 numbers (the 8 bins).

16 sub-regions $\times$ 8 bins = 128 numbers.

The final 128-dimensional descriptor is literally just a list of these numbers: $[v_1, v_2, v_3, ..., v_{128}]$. It tells the computer exactly how edges are spatially arranged around the keypoint.

15.5 Solving Illumination Invariance

What if one image was taken in bright daylight and the other at dusk? The 128 numbers will be drastically different because the gradient magnitudes will be smaller in the dark image. To fix this, we normalize the 128-dimensional vector to have a "unit length" of 1:

$$\hat{v} = \frac{v}{\sqrt{v_1^2 + v_2^2 + ... + v_{128}^2}} \tag{15.1}$$

By doing this, global contrast changes are cancelled out. SIFT also caps any single value at 0.2 and re-normalizes to protect against harsh, non-linear camera saturation.

Chapter 16

Introduction to Feature Matching

Once local features are detected (Chapter 2) and described (Chapter 3), the final step in the pipeline is feature matching: finding the one-to-one correspondences between features in Image 1 and Image 2. This requires defining a **Similarity Measure** (how to score the match mathematically) and a **Matching Strategy** (how to decide which matches to keep).

16.1 Similarity Measures for Image Patches

When using raw image patches of size $W \times W$ (denoted as $N1$ and $N2$) instead of SIFT descriptors, two primary mathematical methods are used to compute their similarity.

16.1.1 Sum of Squared Differences (SSD)

SSD calculates the direct pixel-by-pixel intensity difference:

$$\phi_{SSD}(N1, N2) = -\sum_{k,l=-\frac{W}{2}}^{\frac{W}{2}} (N1(k,l) - N2(k,l))^2 \tag{16.1}$$

The negative sign is applied so that a perfect match results in the maximum possible score (0), while increasingly dissimilar patches yield highly negative scores.

Numerical Example: For two 2×2 patches:

$$N1 = \begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix}, N2 = \begin{bmatrix} 12 & 18 \\ 10 & 20 \end{bmatrix} \; SSD = -[(10-12)^2+(20-18)^2+(10-10)^2+(20-20)^2] = -[4+4+0+0] = \mathbf{-8}$$

16.1.2 Normalized Cross Correlation (NCC)

SSD is highly sensitive to lighting changes. NCC corrects for affine changes in illumination by normalizing the pixels against the patch's mean (μ) and standard deviation (σ):

$$\phi_{NCC}(N1, N2) = \frac{\sum_{k,l=-\frac{W}{2}}^{\frac{W}{2}} (N1(k,l) - \mu_1)(N2(k,l) - \mu_2)}{W^2 \sigma_1 \sigma_2} \tag{16.2}$$

Where μ_i is the mean brightness of patch i, and σ_i is its standard deviation. This function returns a score exactly in the range $[-1, 1]$, where 1 is a perfect match. By subtracting the mean, a bright image and a dark image of the exact same object will perfectly match.

16.2 Similarity Measures for Histograms

When matching SIFT descriptors (which are 128-bin histograms), Euclidean distance is standard, but the course also highlights **Histogram Intersection**:

$$HI(H^1, H^2) = \sum_{i=1}^{N_{bin}} \min(H_i^1, H_i^2) \tag{16.3}$$

This measures the overlapping area between two histograms.

Numerical Example: Let $H^1 = [10, \mathbf{5}, \mathbf{2}]$ and $H^2 = [\mathbf{8}, 20, 3]$. The intersection is $\min(10, 8) + \min(5, 20) + \min(2, 3) = 8 + 5 + 2 = \mathbf{15}$.

16.3 Matching Strategy: Lowe's Ratio Test

A Brute Force approach checks every feature against every other feature and picks the minimum distance. However, in environments with repeating patterns (like windows on a building), multiple features will look mathematically identical, leading to false matches.

David Lowe introduced the **Ratio Test**:

$$Ratio = \frac{\text{Distance to Best Match}}{\text{Distance to Second Best Match}} \tag{16.4}$$

If the ratio is low (e.g., 0.1), the best match is uniquely excellent. If the ratio is high (e.g., 0.9), it means the best match and second-best match are almost identical, indicating an ambiguous match. SIFT strictly rejects any match where this ratio is > 0.8.

16.4 Matching Through an Affinity Matrix

An Affinity Matrix A represents the similarity between all pairs of features, where $A(i, j)$ is the score between feature i in Image 1 and feature j in Image 2. This approach can combine spatial location and visual appearance.

16.4.1 Spatial Vicinity (Distance Matrix E)

$$E(i, j) = e^{-\frac{||p_i^1 - p_j^2||^2}{2\sigma^2}} \tag{16.5}$$

Here, p_i^1 and p_j^2 are the spatial (X, Y) coordinates of the keypoints. σ is a tolerance parameter dictating how much physical movement is allowed. If a point moves minimally, the squared distance $||p_i^1 - p_j^2||^2$ is small, and $e^0 \approx 1$.

16.4.2 Combined Affinity Matrix (A)

To combine the spatial score E with the appearance score NCC, the formula is:

$$A(i, j) = E(i, j) \times \frac{1}{2}(\phi_{NCC}(Q_i^1, Q_j^2) + 1) \tag{16.6}$$

The expression $\frac{1}{2}(NCC + 1)$ shifts the NCC score from $[-1, 1]$ to $[0, 1]$. Therefore, an entry in A will only approach 1.0 if the features are physically close AND look visually identical.

16.4.3 Extracting Matches from A

To guarantee a 1-to-1 correspondence, a match at (i, j) is only accepted if:

1. $A(i, j)$ is the maximum value in row i.
2. $A(i, j)$ is the maximum value in column j.
3. $A(i, j) >$ threshold.

Chapter 17

Introduction to Motion Analysis

Motion analysis involves analyzing video sequences rather than static images to focus on motion information. This adds a temporal dimension, allowing the visual system to perceive structures and details that might be invisible in a single frame.

17.1 The Image Sequence

An image sequence is defined as a series of N images, or frames, acquired at discrete time instants. The mathematical representation of frame timing is:

$$t_k = t_0 + k\Delta t \tag{17.1}$$

Where:

- t_k: Time of the current frame k.
- t_0: The starting time.
- Δt: The fixed (usually small) time interval between acquisitions (e.g., 1/30s for standard video).

17.2 Brightness Constancy Assumption

A fundamental assumption in motion analysis is that the illumination of the scene does not vary significantly within the observation interval. Under this assumption, changes in pixel intensity from time t_1 to t_2 are attributed solely to the relative motion between the scene and the camera.

17.3 Core Problems in Motion Analysis

Motion analysis can be categorized into several distinct computational problems:

- **Correspondence**: Identifying which elements of one frame correspond to which elements of the next frame. This is closely related to image matching.
- **Reconstruction**: Determining the 3D position and 3D velocity of observed elements from their 2D motion on the image plane.
- **Motion Segmentation**: Partitioning the image into regions corresponding to different moving parts.
- **Tracking**: Estimating the trajectories (2D or 3D) of specific points or objects over a sequence of frames using tools like dynamic filters (e.g., Kalman filters).

17.4 Methodological Summary

The course focuses on the following low-level algorithms and temporal features:

1. **Change Detection**: Used for motion segmentation when the camera is stationary.
2. **Optical Flow**: Addressing the correspondence problem to estimate pixel-wise motion.
3. **Feature Tracking**: Following specific image points (like corners) through the sequence.

Chapter 18

Motion Segmentation: Change Detection

When the camera is stationary, motion segmentation is implemented as a difference between the current frame and a reference model.

18.1 Basic Thresholding

The threshold τ is used to separate motion from noise.

$$M_t(x,y) = \begin{cases} 1 & \text{if } |I_t(x,y) - I_{ref}(x,y)| > \tau \\ 0 & \text{otherwise} \end{cases} \tag{18.1}$$

The threshold τ depends on acquisition conditions and sensor noise.

18.1.1 Statistical Basis

In an empty scene, the difference $|I_t - I_{t+1}|$ follows a Quasi-Gaussian distribution caused by sensor noise and illumination instability (e.g., neon flickering).

- **Stable Conditions**: Narrow distribution. Typically $\tau \approx 10$.
- **Unstable Conditions**: Wide distribution. Typically $\tau \approx 30$.

18.2 Background Modeling Versions

18.2.1 Version 1: Static Average

The reference image is the average of the first N frames:

$$B = \frac{1}{N} \sum_{t=1}^{N} I_t \tag{18.2}$$

Numerical Example: For $N = 2$, if a pixel (x, y) has intensities $I_1 = 100$ and $I_2 = 110$, then $B = 105$. This value is fixed.

18.2.2 Version 2: Running Average (Temporal Filtering)

Handles slow illumination changes by updating the background model B_t at each frame using a learning rate α:

$$B_t = (1 - \alpha)B_{t-1} + \alpha I_t \tag{18.3}$$

Numerical Example: If $B_{t-1} = 100$, $I_t = 120$, and $\alpha = 0.05$: $B_t = (0.95 \times 100) + (0.05 \times 120) = 95 + 6 = 101$.

18.2.3 Version 3: Selective Update for Abrupt Variations

Designed to handle persistent changes (e.g., a moved object). It compares the current frame to a frame from Δt steps ago to check for persistence.

$$B_t = \begin{cases} B_{t-1} & \text{if } |I_t - I_{t-\Delta t}| > \tau' \\ (1-\alpha)B_{t-1} + \alpha I_t & \text{otherwise} \end{cases} \tag{18.4}$$

Numerical Example: A person walks through a pixel. $|I_t - I_{t-\Delta t}|$ is high (e.g., 50). The background is NOT updated ($B_t = B_{t-1}$), preventing the "ghosting" of the person into the background.

18.3 Blob Analysis and Descriptors

After thresholding, pixels are grouped into **Blobs** via Connected Components. A blob is a segment where any two pixels can be connected by a path remaining inside the segment.

18.3.1 Common Descriptors

- **Centroid $(\bar{x}, \bar{y})$**: $\bar{x} = \frac{1}{A}\sum x_i, \bar{y} = \frac{1}{A}\sum y_i$, where A is Area.
- **Area**: Total number of pixels in the blob.
- **Velocity**: $\vec{v} = (v_x, v_y)$, representing the displacement magnitude and direction.
- **Higher Order Moments**: Used for shape description and orientation.

18.4 Recap

1. **Static Average**: $B = \frac{1}{N}\sum I_t$. (Assumes constant background).
2. **Running Average**: $B_t = (1-\alpha)B_{t-1} + \alpha I_t$. (Handles slow illumination changes).
3. **Selective Persistent Update**: Only updates the background if the change is persistent ($|I_t - I_{t-\Delta t}|$ is small), preventing moving objects from being incorporated into the background.

Chapter 19

Correspondence: Optical Flow

Correspondence identifies matching elements across frames acquired close in time ($t_k = t_0 + k\Delta t$). This allows us to calculate **Optical Flow**: the apparent motion of brightness patterns.

19.1 The Concept of Optical Flow

In the context of motion, correspondence can be viewed as estimating the **apparent motion** of the brightness pattern, known as **Optical Flow**.

Why apprant? Optical flow is the motion we perceive, which may not always match the true 3D physical motion of the object. The slides provide two classic examples:

- **The Bouncing Ball**: A uniform, textureless ball bouncing in a dark room. Because the pixels don't change color or intensity as the ball moves, the visual system perceives nothing—the optical flow is zero despite the physical motion.
- **The Barber Pole**: As a barber pole rotates (3D rotational motion), we perceive the stripes moving vertically (apparent motion). This illustrates the difference between the motion field (the 2D projection of 3D motion) and the optical flow.

19.2 Correlation-Based Approaches

This is a "search-based" method for finding correspondence. Instead of using calculus, we look for matching pixel patterns. **The Algorithm:**

1. **Select a Feature**: Identify a meaningful point (like a corner) in image I_t and define a surrounding "patch" or neighborhood N_1.
2. **Define a Search Region**: Define a small region R in the next frame I_{t+1} where you expect the point to have moved.
3. **Find the Best Match**: Slide the patch N_1 over the search region R and compare it to candidate patches N_2 using a similarity measure.
4. **Estimate Motion**: The distance between the original position of N_1 and the position of the most similar patch N_2^* is the local estimate of apparent motion.

19.2.1 Similarity Measures

We measure how "close" two patches are pixel-by-pixel using mathematical functions. Two common measures are:

1. Sum of Squared Differences (SSD):

$$\phi_{SSD}(N1, N2) = -\sum_{k,l}(N1(k,l) - N2(k,l))^2 \tag{19.1}$$

- **Logic**: We subtract the intensity of each pixel in N_2 from N_1, square the result (to make all differences positive), and sum them.
- **Goal**: The best match is the one where the sum is closest to zero (represented here with a negative sign so that a "higher" value near 0 is better).

2. Normalized Cross Correlation (NCC):

$$\phi_{NCC}(N1, N2) = \sum_{k,l} \frac{(N1(k,l) - \mu_1)(N2(k,l) - \mu_2)}{W^2 \sigma_1 \sigma_2} \tag{19.2}$$

- μ **(Mean)**: The average brightness of the patch.
- σ **(Standard Deviation)**: The contrast or variation in the patch.
- **Logic**: By subtracting the mean, this measure is robust to global lighting changes (e.g., the whole image getting slightly brighter).
- **Range**: The result is always between $[-1, 1]$, where 1 is a perfect match.

19.2.2 Numerical Example: SSD Calculation

Given patch $N_1 = [10, 20; 10, 20]$ and candidate $N_2 = [12, 22; 12, 22]$:

1. Differences: $(10 - 12) = -2, (20 - 22) = -2, \ldots$
2. Squares: $(-2)^2 = 4$
3. Sum: $4 + 4 + 4 + 4 = 16$
4. $\phi_{SSD} = -16$

19.3 Gradient-Based Optical Flow

The Brightness Constancy Equation assumes $I(x, y, t) = I(x + u, y + v, t + 1)$. By Taylor expansion, we obtain the **Optical Flow Constraint Equation**:

$$I_x u + I_y v + I_t = 0 \tag{19.3}$$

where I_x, I_y are spatial gradients and I_t is the temporal gradient.

19.3.1 The Aperture Problem

One equation with two unknowns (u, v) is underdetermined. We can only compute motion in the direction of the gradient (**Normal Flow**):

$$u_n = \frac{-I_t}{\sqrt{I_x^2 + I_y^2}} \tag{19.4}$$

19.4 The Lucas-Kanade Algorithm

By assuming velocity $\mathbf{u}$ is constant over a neighborhood N of m pixels, we solve the overdetermined system $A\mathbf{u} = \mathbf{b}$ using the pseudo-inverse:

$$\mathbf{u} = (A^\top A)^{-1} A^\top \mathbf{b} \tag{19.5}$$

where:

$$A = \begin{bmatrix} I_x(p_1) & I_y(p_1) \\ \vdots & \vdots \\ I_x(p_m) & I_y(p_m) \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -I_t(p_1) \\ \vdots \\ -I_t(p_m) \end{bmatrix} \tag{19.6}$$

19.5 Multi-Resolution Pyramids

To handle large displacements where Taylor approximations fail, we use **Gaussian Pyramids**.

1. Estimate flow at a coarse (downsampled) level.
2. Upsample the flow to the next level: $u(i) = 2 \times u(i-1)$.
3. **Warp** frame $I(t)$ using the current estimate.
4. Calculate the local correction $u'(i)$ using LK on the warped image.
5. Final Flow: $u_{final} = \sum u_{corrections}$.

19.5.1 Numerical Examples

1. LK System: If $A^\top A = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$ and $A^\top \mathbf{b} = \begin{bmatrix} 20 \\ 50 \end{bmatrix}$, then: $\mathbf{u} = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix} \begin{bmatrix} 20 \\ 50 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$.

2. Pyramid Upsampling: If a pixel moves 3 pixels at level $i = 1$ (half resolution), it corresponds to a 6-pixel move at level $i = 0$ (full resolution).

Chapter 20

Correspondence Over Time: Feature Tracking

While optical flow calculates motion between two consecutive frames, **Feature Tracking** stitches these frame-to-frame movements together to form long-term trajectories across a whole video sequence.

20.1 The Multi-Frame Tracking Algorithm

The basic algorithm tracks salient points (like corners) through the sequence:

1. **Initialization**: Given a video sequence $I(0), I(1), \ldots, I(T)$, extract a set of corners c_i from the initial frame $I(0)$ for $i = 1, \ldots, N$.
2. **Update Loop**: For each pair of adjacent frames $I(t)$ and $I(t+1)$:
    - For each tracked corner $c_i(t)$ on $I(t)$, estimate its optical flow (using methods like Lucas-Kanade) to find its updated position $c_i(t+1)$.
    - Update the trajectory record for c_i.

Output: A set of N corner tracks detailing the path of each point over time.

Numerical Example: A corner is detected at $c_1(0) = (10, 10)$. Between $t = 0$ and $t = 1$, optical flow estimates $\mathbf{u} = (1, -1)$. New position: $c_1(1) = (11, 9)$. Between $t = 1$ and $t = 2$, optical flow estimates $\mathbf{u} = (2, 0)$. New position: $c_1(2) = (13, 9)$. The track for c_1 is $[(10, 10), (11, 9), (13, 9)]$.

20.2 The Kalman Filter in Tracking

Because optical flow estimates are noisy and sometimes fail (due to occlusion or blurring), we use a **Kalman Filter** to maintain robust tracks.

20.2.1 Key Roles of the Kalman Filter

- **Smoothing**: It smooths out noisy trajectories by blending the actual measurements with a physical prediction model (e.g., constant velocity).
- **Bridging Gaps**: If a feature is temporarily lost (occluded), the filter uses the estimated state (prediction) as a hypothesis to fill in the missing measurement.
- **Uncertainty Tracking**: It maintains a state covariance matrix P_t, which provides a measure of uncertainty, often visualized as an uncertainty ellipse around the predicted position.

20.3 The Data Association Problem

When tracking multiple targets simultaneously, the predictions of where targets will be in the next frame can overlap.

If Track 1 and Track 2 are close to each other, their prediction ellipses might intersect. When a new corner is detected in that intersection area, the system faces the **Data Association Problem**: determining which track the newly detected measurement actually belongs to.

Solutions typically involve calculating distances (like the Mahalanobis distance, which accounts for the shape of the uncertainty ellipse P_t) and assigning the measurement to the most probable track.

Chapter 21

Principles of 3D computer vision: Introduction

Before diving into complex math to extract 3D data from 2D images (passive vision), it is important to understand **Active 3D Sensors**. These are hardware solutions that actively emit energy into the environment to measure distance:

- **Time-of-Flight (ToF):** These sensors measure the exact time it takes for a light pulse (like a laser) to bounce off an object and return to the photodetector.
- **LIDAR / RADAR / IR:** LIDAR uses lasers for short/medium range, RADAR uses radio frequencies for medium/long range, and IR (Infrared) is often used for short-range depth cameras.

21.1 The Geometry of Image Formation

When using standard cameras, we model how a 3D world gets flattened onto a 2D sensor using the **perspective projection** (or pinhole camera model).

- **The 3D World Point:** We define a point in the real world using 3D coordinates relative to the camera: $P = (X, Y, Z)$.
- **The 2D Image Point:** This 3D point projects onto the camera's image plane (the digital sensor) at a specific 2D pixel coordinate, located at a distance f (the focal length) from the camera's optical center: $p = (x, y, f)$.

The mathematical projection is based on similar triangles:

$$x = f\frac{X}{Z}$$
$$y = f\frac{Y}{Z}$$

Numerical Example:
Imagine your camera has a focal length $f = 50$ mm. You are looking at an object located 1000 mm to the right ($X = 1000$), 500 mm up ($Y = 500$), and 5000 mm away from you in depth ($Z = 5000$). Where does it appear on the camera sensor?

- $x = 50 \cdot (1000/5000) = 50 \cdot 0.2 = 10$ mm
- $y = 50 \cdot (500/5000) = 50 \cdot 0.1 = 5$ mm

The object will be projected onto the sensor 10 mm to the right and 5 mm up from the center.

21.2 The Core Problem: Scale Ambiguity

The fundamental flaw of single-camera 2D vision is **Scale Ambiguity**. Because the perspective projection is a "lossy" and "non-invertible" transformation (meaning depth information Z is lost during the projection), the camera cannot tell the difference without a prior knowledge between a small object up close and a massive object far away.

Revisiting the math example: $X/Z = 1000/5000 = 0.2$.
What if the object was twice as big and twice as far away? ($X = 2000, Z = 10000$). The ratio $2000/10000$ is still 0.2. The object projects to the exact same 10 mm spot on the sensor!

21.3 The Solution: A Second View

To recover the lost depth, we add a second camera. By capturing the same 3D point P from two different angles, it will project to point p on the first image plane and p' on the second image plane. Comparing the difference between these two 2D points allows us to compute the original depth.

Chapter 22

Stereopsis

Stereo vision (or stereopsis) is the process of inferring 3D structure and distance from two or more images taken from different viewpoints. This field of computer vision is heavily inspired by human biology. Because our eyes are set slightly apart, each eye gets a slightly different view of the world. Our brain perceives 3D depth by calculating the difference in the retinal position of the objects we see. This difference is called **disparity**.

22.1 The Math of a Simple Stereo System

Imagine two identical cameras placed side-by-side, perfectly aligned, looking straight ahead.

- f: The focal length of both cameras.
- T: The "baseline," which is the physical physical distance between the optical centers of the two cameras.
- x_l and x_r: The horizontal pixel coordinates where a specific 3D point P appears on the left and right image sensors, respectively.

The difference between these two points is the **disparity**: $d = |x_l - x_r|$. Depth (Z) is calculated using the following formula:

$$Z = \frac{fT}{d} \tag{22.1}$$

The Golden Rule of Stereo Vision: Depth is *inversely* proportional to disparity.

- Large disparity = The object is very close.
- Small disparity = The object is far away.
- Zero disparity = The object is infinitely far away (like the horizon).

Numerical Example:
Let's say you have a stereo camera rig where the lenses are $T = 100$ mm apart, and both have a focal length of $f = 50$ mm. You take a picture of an object. On the left camera sensor, the object is at pixel coordinate $x_l = -10$ mm. On the right camera sensor, it is at $x_r = -30$ mm.

- Disparity: $d = |-10 - (-30)| = 20$ mm.
- Depth: $Z = \frac{50 \cdot 100}{20} = \frac{5000}{20} = 250$ mm.

22.2 Geometric Derivation of the Depth Equation

Why does $Z = fT/d$ work? It is based on the exact same similar triangles principle from the pinhole camera model, applied twice.

Imagine looking at the camera setup from a top-down view:

1. Let the **left camera's optical center** be at the origin $(0, 0)$.

2. Let the **right camera's optical center** be shifted to the right by the baseline T, so it sits at $(T, 0)$.

3. A 3D point P exists in the real world at coordinates (X, Z).

Left Camera Projection:
Using similar triangles, the light ray from P to the left camera intersects the image plane (at distance f) at coordinate:

$$x_l = f\frac{X}{Z} \tag{22.2}$$

Right Camera Projection:
Because the right camera is physically moved to the right by T, the relative X coordinate of the point P from the perspective of the right camera is $(X - T)$. Using similar triangles again, the projection on the right image plane is:

$$x_r = f\frac{X - T}{Z} \tag{22.3}$$

Calculating Disparity:
Now, we define disparity d as the difference between the left and right image coordinates:

$$\begin{aligned} d &= x_l - x_r \\ d &= \left(f\frac{X}{Z}\right) - \left(f\frac{X-T}{Z}\right) \\ d &= \frac{fX - f(X-T)}{Z} \\ d &= \frac{fX - fX + fT}{Z} \\ d &= \frac{fT}{Z} \end{aligned}$$

Solving for Z, we get our final depth equation:

$$Z = \frac{fT}{d} \tag{22.4}$$

This beautifully shows that the X coordinate (lateral position) cancels out entirely! Depth depends only on the camera properties (f, T) and the measured disparity (d).

22.3 The Two Main Problems of Stereopsis

While the formula is simple, making a computer execute it is difficult. Stereo vision covers two main problems:

1. **The Correspondence Problem:** Given a specific pixel in the left image, how do we find the *exact same* pixel in the right image? Solving this produces dense disparity maps.

2. **The 3D Reconstruction Problem:** Once we have all the disparities, how do we translate them back into the real world to build a full 3D point cloud?

Chapter 23

Correspondence Problem

To calculate depth using stereo vision, we must first determine the disparity ($d = x_l - x_r$). The **Correspondence Problem** is the challenge of figuring out which pixel in the right image corresponds to a specific pixel in the left image. To solve it, we must decide which image elements to match and which similarity measure to use.

23.1 Sparse vs. Dense Correspondences

We can approach matching in two ways:

- **Sparse Correspondences:** We only try to match highly distinct features, like corners or sharp edges. This is faster but only provides depth for a few specific points.
- **Dense Correspondences:** We attempt to find a match for *every single pixel* in the image to create a full "disparity map". In a standard color-coded disparity map, bright areas represent high disparities (objects closer to the camera), and dark areas represent low disparities (objects farther away).

23.2 Image Rectification

Searching an entire 2D image for a matching pixel is computationally massive. To simplify this, we mathematically warp the stereo pair of images in a process called **Rectification**. Rectification perfectly aligns the two cameras virtually so that their image planes are coplanar.

Because of this geometry, a point on row y in the left image will *always* lie on the exact same row y in the right image (conjugate points lie on corresponding scanlines). Instead of searching a whole 2D image, we only have to search a single 1D horizontal line!

23.3 The Block Matching Algorithm (Dense Correspondences)

To find matches, we cannot simply compare single pixels, as many pixels share the same color/intensity. Instead, we use correlation methods applied to small image patches (a neighborhood or window of pixels).

Algorithm Sketch:

1. Take a small window W (e.g., a 5×5 pixel box) centered on a pixel at coordinates (i, j) in the left image I_l.
2. Slide an identical window along the corresponding row in the right image I_r, shifting it by a disparity distance d. We restrict d to a specific search range $[d_{min}, d_{max}]$.
3. Calculate a similarity score $c(d)$ between the two windows at each step. Common metrics include **SSD** (Sum of Squared Differences) or **NCC** (Normalized Cross-Correlation).
4. The true disparity $\overline{d}$ is the shift that yields the best similarity score (e.g., the minimum difference for SSD).

Numerical Example: Sum of Squared Differences (SSD)
Let's look at a 1D example. We have a small window of 3 pixels in the left image: $[4, 8, 5]$. We are searching along a row in the right image: $[1, 4, 8, 5, 2]$.

- **Test $d = 0$ (shift by 0):** Compare $[4, 8, 5]$ with $[1, 4, 8]$.
  SSD $= (4-1)^2 + (8-4)^2 + (5-8)^2 = 9 + 16 + 9 = 34$
- **Test $d = 1$ (shift by 1):** Compare $[4, 8, 5]$ with $[4, 8, 5]$.
  SSD $= (4-4)^2 + (8-8)^2 + (5-5)^2 = 0 + 0 + 0 = 0$

Since we want the *minimum* difference, $d = 1$ is our winner. The disparity is 1 pixel.

23.4 Left-Right Consistency and Occlusions

Correspondences are made more difficult by **occlusions** (points in the 3D world that are visible to one camera but hidden behind an object from the perspective of the other camera). A point with no counterpart will break the matching algorithm.

To solve this, we enforce **Left-Right Consistency**:

1. Compute a disparity map from the left image to the right image (D_{lr}).
2. Compute a second disparity map from the right image to the left image (D_{rl}).
3. A match is considered valid if the left-to-right disparity is equal and opposite to the right-to-left disparity:
$$D(i, j) = d \iff D_{lr}(i, j) = -D_{rl}(i, j + d) = d \tag{23.1}$$

If they disagree, we mark that pixel as an occlusion.

Chapter 24

Understanding Image Semantics

24.1 What does "understanding the semantics of an image" mean?

A digital image, to a computer, is nothing more than a grid of numbers. A grayscale image is a matrix of intensity values (typically integers in $[0, 255]$); a colour image is three such matrices stacked together (the Red, Green and Blue channels). *Semantics* refers to the **meaning** we humans attach to that grid: "this is a face", "this is a sailboat", "this pixel belongs to the sky".

The central problem of this part of the course is to bridge the gap between the *raw signal* (numbers) and the *semantic content* (labels, objects, regions). The simplest version of this bridge is **image classification**.

Key idea

Image classification = associate a single label y to a whole image, chosen from a fixed set of possible labels Y. It is the entry point to every more complex task (detection, segmentation), because all of them reduce, in some way, to classifying a region or a pixel.

24.2 The supervised learning formulation

To learn a classifier we collect a **training set** of n examples, each being an (input, output) pair:

$$(\mathbf{x}_1, y_1),\ (\mathbf{x}_2, y_2),\ \ldots,\ (\mathbf{x}_n, y_n), \qquad \mathbf{x}_i \in \mathbb{R}^p, \quad y_i \in Y, \quad i = 1, \ldots, n.$$

Let us unpack every symbol, because this notation reappears constantly.

- $\mathbf{x}_i \in \mathbb{R}^p$ is the **feature vector** (or *representation*) of the i-th image. The image has been turned into a list of p real numbers. In the simplest case those numbers are just the pixel intensities read out one after another; in modern systems they are the output of a neural network. The number p is the **dimensionality** of the representation.
- $y_i \in Y$ is the **label** of the i-th image. The set Y might be $\{-1, +1\}$ (binary), $\{1, 2, \ldots, K\}$ (K classes), etc.
- n is the **number of training examples**; p is the **number of features per example**. Keep these two apart — confusing "how many images" with "how big is each image vector" is a classic source of errors.

24.2.1 Stacking the data into matrices

It is convenient to stack all the inputs into one **data matrix** X_n and all the labels into one vector Y_n:

$$X_n = \begin{pmatrix} x_1^1 & x_1^2 & \cdots & x_1^p \\ x_2^1 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \ddots & \vdots \\ x_n^1 & x_n^2 & \cdots & x_n^p \end{pmatrix} \in \mathbb{R}^{n \times p}, \qquad Y_n = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n.$$

Key idea

Row = one image, Column = one feature. Row i of X_n is the whole representation $\mathbf{x}_i$ of image i. The superscript indexes the feature (x_i^j is the j-th feature of the i-th image). So X_n has n rows and p columns.

Numeric example: reading out a tiny image

Suppose each image is a 2×2 grayscale patch. Flattening it row-by-row gives $p = 4$ features. Take the patch

$$\begin{pmatrix} 170 & 238 \\ 85 & 0 \end{pmatrix} \longrightarrow \mathbf{x} = (170,\ 238,\ 85,\ 0) \in \mathbb{R}^4.$$

If we have $n = 3$ such patches, the data matrix is 3×4:

$$X_n = \begin{pmatrix} 170 & 238 & 85 & 0 \\ 68 & 136 & 17 & 170 \\ 221 & 0 & 238 & 136 \end{pmatrix}.$$

A realistic 256×256 grayscale image instead gives $p = 256 \times 256 = 65\,536$ features — this is why image representations quickly become very high-dimensional, and why we look for smarter, more compact representations than raw pixels.

24.3 Binary classification: the "face / not face" problem

The simplest classification task has only two labels. The slides use the classic example of a **face detector**, with the encoding

$$y_i = \begin{cases} -1 & \text{if image } i \text{ is a face,} \\ +1 & \text{if image } i \text{ is } \textit{not} \text{ a face.} \end{cases}$$

(The particular choice of $+1/-1$ and which class gets which sign is a convention; what matters is that the two classes are separated by a sign.)

The learning procedure is:

1. collect many image patches, each turned into a feature vector $\mathbf{x}_i$ (a row of X_n) and labelled ± 1;
2. feed (X_n, Y_n) to a learning algorithm, which searches for a **decision rule** (a curve / surface in $\mathbb{R}^p$, sketched as the green line in the slide) separating the "face" points from the "not-face" points;
3. at the end of **training** and **validation** we obtain a *model* $f : \mathbb{R}^p \to \{-1, +1\}$ that, given a *new* unseen patch, predicts whether it is a face.

Training vs. validation (in one line each)

Training = the data the algorithm uses to fit the model's parameters. **Validation** = a separate held-out chunk of data used to tune choices (e.g. model complexity) and to estimate how well the model will generalise to new images. Reporting accuracy on data the model has already seen during training is misleading — this is why the two sets are kept distinct.

24.4 Multi-class classification: the label set grows

Reality is rarely binary. As soon as we want to distinguish many categories — *cat, dog, boat, car, ...* — we move to **multi-class classification**, where $Y = \{1, 2, \ldots, K\}$ with K classes.

A subtlety the slides highlight (with the *mammal* → *placental* → *carnivore* → *canine* → *dog* → *working dog* → *husky* chain) is that semantic labels are not flat: they form a **hierarchy / taxonomy**. The same image of a husky is correctly labelled "animal", "dog", or "husky" — at different levels of granularity. This is exactly the structure that the large-scale dataset ImageNet exploits (it is built on the WordNet hierarchy), which we will meet again when we discuss CNNs.

Key idea

Granularity is a design choice. "How many classes, and how fine?" is decided by the problem, not by the images. Coarser label sets are easier to learn but less informative; finer sets are more useful but need more data and a more discriminative representation.

Numeric example: why more classes is harder

A *random* binary classifier is right $1/2 = 50\%$ of the time. A random classifier over $K = 1000$ classes (the ImageNet challenge size) is right only

$$\frac{1}{K} = \frac{1}{1000} = 0.1\%$$

of the time. The "baseline you must beat" collapses as K grows, so the representation $\mathbf{x}$ has to be far more descriptive to separate 1000 classes than to separate 2. This is the quantitative reason multi-class problems are so much more demanding.

24.5 Classification, Object Recognition and Instance Recognition

The slides put three closely related ideas side by side. They sound almost synonymous in everyday speech, but in computer vision they denote different tasks with different inputs, outputs and difficulties. This short chapter pins down the distinctions, because the rest of the course (detection, segmentation) builds on getting them straight.

24.5.1 Image classification vs. object recognition

Recall from Section 24.1 that **image classification** assigns one label to the *whole* image. Nothing in that definition requires the image to contain a discrete "thing".

- An autumn avenue of trees can be classified as `forest` / `autumn` / `landscape`. There is no single object — the label describes the *scene as a whole*, its overall texture and colour distribution.
- A photo of a puppy can also be classified (`dog`), but here the label is driven by a **main object** occupying the image. In this second situation we could equally well say we are doing **object recognition**.

Key idea

The two coincide when the image is dominated by one object. Image classification asks "what is this a picture *of*?" and is happy with scenes, textures or objects alike. Object recognition specifically asks "*which object* is shown?", and only makes sense when there is an object to name. Whenever a single object fills the frame, classifying the image and recognising the object are effectively the same operation.

The practical consequence: an object-recognition problem in which each image contains exactly one centred object *is* a classification problem, and can be attacked with the exact machinery of the previous sections (feature vector $\mathbf{x}_i$, label y_i, decision rule). The harder case — many objects, anywhere in the image — is **object detection**, the subject of a later chapter.

24.5.2 Object (category) recognition vs. instance recognition

The second, more subtle distinction concerns *how specific* the label is. Consider the difference between the sentences "that is a mug" and "that is *my* mug".

Category recognition vs. instance recognition

Category (generic object) recognition: decide that an image contains *some* member of a class — "a car", "a bicycle", "a mug". The correct answer is the same for every car ever built.

Instance recognition: decide that an image contains *one particular* object — "*his* car", "*that* bicycle", "*my* mug". The correct answer distinguishes this specific physical object from all other members of the same category.

The slides illustrate this with a collection of mugs. *Category recognition* must say "mug" for all of them — the black "BUT FIRST COFFEE" mug, the white one, the steel travel mug, the painted teacup, the blue one and the yellow one are all positive examples of the single class `mug`. *Instance recognition* must instead single out one specific mug — e.g. the purple "Francesca" mug — and recognise *that one*, ignoring all the others even though they belong to the same category.

Why this matters: the two tasks pull in opposite directions

The distinction is not merely terminological; it dictates what the representation $\mathbf{x}$ must be good at.

- Category recognition needs a representation that is **invariant** to within-class differences. Colour, shape details, brand and lighting all change from one mug to another, yet the answer "mug" must stay the same. The representation should *discard* the things that distinguish individual mugs and keep only what makes a mug a mug.
- Instance recognition needs a representation that is **sensitive** to exactly those within-class differences. The text "Francesca", the precise purple, the particular shape are now the *signal*, not the noise. Here the representation should *preserve* fine, object-specific detail.

Key idea

Invariance is task-dependent. The very features that a category recogniser should ignore (the ones telling mugs apart) are the features an instance recogniser must rely on. There is no single "best" image representation in the abstract — "best" only makes sense relative to the task and to *which variations should be ignored*. This idea — choosing what to be invariant to — is the thread running through the next chapter on image representations.

Numeric example: counting positives in the mug collection

Suppose a shelf photo contains 6 mugs, of which exactly 1 is the "Francesca" mug.

- For **category recognition** of the class `mug`: all 6 mugs are positive examples ($y = +1$) and everything else (table, wall, etc.) is negative. Positives = 6.
- For **instance recognition** of the specific Francesca mug: only 1 object is positive; the other 5 mugs — despite being mugs — are now *negatives*, together with the rest of the scene. Positives = 1.

The same 6 objects, the same pixels, but the label vector Y_n changes completely depending on whether we are doing category or instance recognition.

24.5.3 Summary of the three tasks

Task	Question it answers	Output granularity
Image classification	"What is this a picture of?" (scene or object)	One label for the whole image
Object (category) recognition	"Which *kind* of object is shown?"	A class shared by all members
Instance recognition	"Which *specific* object is shown?"	A single physical object

All three remain *whole-image* tasks: they output one answer for the image as a whole. The moment we allow several objects at unknown positions, we step into detection and segmentation — which is exactly where the course goes next.

24.6 Image Representations

24.6.1 What is an image representation, exactly?

We have repeatedly written $\mathbf{x}_i \in \mathbb{R}^p$ for "the i-th image as a vector". This chapter asks the question the slides pose bluntly — *what are they, exactly?* — and answers it.

An **image representation** (also called a **feature vector** or **descriptor**) is a function

$$\phi : (\text{images}) \longrightarrow \mathbb{R}^p, \qquad \mathbf{x}_i = \phi(\text{image}_i),$$

that turns a raw image into a fixed-length vector of p numbers. Everything downstream — the data matrix X_n, the decision rule, the accuracy — depends on this ϕ. Choosing ϕ is arguably *the* central design decision in classical computer vision: the same learning algorithm can succeed or fail entirely depending on how the images were turned into vectors.

Key idea

The representation is the bridge. Learning algorithms operate on $\mathbb{R}^p$, not on pixels. The map ϕ decides which information survives into $\mathbf{x}$ and which is thrown away. A good ϕ makes the subsequent classification easy (classes well separated in $\mathbb{R}^p$); a bad ϕ can make it impossible no matter how powerful the classifier.

24.6.2 Two requirements a good representation must satisfy

The slides state two demands, which are worth reading carefully because they are in tension with one another.

1. **It must describe the whole image** (or the whole region we are about to classify). The vector $\mathbf{x}$ should summarise all the relevant content, not just an arbitrary corner.
2. **It must be robust to intra-class variations** of many different kinds. Two images of the same class can look very different at the pixel level, yet ϕ should map them to *nearby* points in $\mathbb{R}^p$ so that the classifier still groups them together.

The seven sources of intra-class variation

The slide lists the standard catalogue of nuisances — the ways two images of the *same* category can differ. A representation that is invariant (or at least robust) to these is what we are after.

The catalogue of variations

- **Viewpoint variation:** the same object photographed from different angles (the statue seen frontally vs. from the side). The pixel arrangement changes completely while the identity does not.
- **Scale variation:** the object appears larger or smaller (close-up vs. far away). Same object, different number of pixels.
- **Deformation:** non-rigid objects change shape (a cat curled up vs. stretched out). The category is stable, the geometry is not.
- **Occlusion:** only part of the object is visible (a cat hidden behind a table, only its head poking out). The representation must cope with *missing* information.
- **Illumination conditions:** the same scene under different lighting (bright, dim, coloured light, shadows). Pixel intensities shift dramatically even though nothing in the scene changed.
- **Background clutter:** the object sits in a busy, distracting background (a spotted cat against a similarly spotted carpet). The representation must separate object from context.
- **Intra-class variation:** genuine diversity *within* the category itself (chairs come in countless shapes, colours and styles, yet all are "chairs"). This is variation of the object identity, not of the imaging conditions.

Key idea

The fundamental tension. Requirement 2 pushes us to *discard* information (be invariant to viewpoint, lighting, ...), while requirement 1 pushes us to *keep* information (describe the whole image). Designing ϕ is the art of throwing away the nuisance variation while preserving the discriminative content. As noted at the end of Chapter 2, *which* variation counts as nuisance depends on the task: viewpoint is nuisance for "is this a chair?" but signal for "which way is the chair facing?".

Numeric example: why raw pixels are not robust to illumination

Take a tiny 1-pixel "image" of a grey patch with intensity $x = 100$. Photograph the identical patch under a light that is twice as bright: every intensity scales by 2, so now $x' = 200$. As raw-pixel "representations" these two are

$$|x' - x| = |200 - 100| = 100,$$

i.e. as far apart (on the 0–255 scale) as black is from mid-grey — even though it is *the same patch*. A representation based directly on pixel values treats a mere lighting change as a huge difference. A representation based instead on, say, the *ratio* or the *gradient* of intensities would be unaffected: $\frac{\partial}{\partial u}(2x)$ and $\frac{\partial}{\partial u}(x)$ differ only by a constant factor, and a normalised gradient cancels it entirely. This is the concrete reason pixels score "not so robust" in the list below.

24.6.3 A ladder of (global) image representations

The slide gives a short list that doubles as a *historical and conceptual progression*, from naive to powerful. The phrase "the list of possibilities is huge" is the honest caveat: these are representative rungs, not an exhaustive enumeration. The word **global** means the representation describes the whole image (or region) with a single vector, as opposed to a *local* description of one small patch.

1. **Pixels** — *(not so robust).* Use the raw intensities as $\mathbf{x}$. Maximally complete (nothing is discarded) but, as the example above shows, hopelessly sensitive to illumination, viewpoint, scale, ... and extremely high-dimensional (p = number of pixels). Almost never used directly.
2. **Graylevel / colour / gradient histograms** — *(not too descriptive).* Summarise the image by *counting* how often each intensity, colour or gradient orientation occurs, ignoring *where* it

occurs. This buys robustness (a histogram is invariant to where things are, and largely to small deformations and translations) at the price of discriminative power: two very different images can share the same histogram. *Robust but blurry.*

3. **Local keypoints in a Bag of Keypoints** — *(still too hand-crafted).* Detect interesting local patches (corners, blobs), describe each one, and pool them into a global vector (the Bag-of-Words idea, next chapter). Far more descriptive and robust than histograms, and the dominant approach before deep learning — but the detectors and descriptors are *designed by hand*, so they encode the engineer's guesses about what matters rather than what the data says.

⋮

4. **CNN features** — *(learnt, supervised or unsupervised).* Instead of hand-designing ϕ, *learn* it from data. A convolutional neural network discovers, layer by layer, the features that best serve the task. This is the leap from *feature engineering* to *feature learning*, and it is why CNNs displaced the earlier rungs.

Key idea

The trajectory of the whole field. Reading the list top to bottom we move along two axes simultaneously: *increasing robustness + descriptiveness*, and *decreasing human hand-design*. Pixels are fully hand-given and fragile; histograms add a little robustness; bag-of-keypoints adds a lot but is still hand-crafted; CNN features are *learnt from the data itself*. The next two chapters follow exactly this path: first Bag-of-Words (the peak of hand-crafted methods), then CNNs (the move to learned representations).

Numeric example: a histogram throws away location

Consider two 2×2 binary images (pixel values in $\{0, 1\}$):

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Their graylevel histograms each count "two 0s and two 1s":

$$h(A) = (2,\, 2) = h(B).$$

The histograms are *identical*, so a histogram-based representation cannot tell A from B — yet the images are clearly different (the diagonal is flipped). This is precisely the "not too descriptive" weakness: by counting values and discarding their positions, the histogram gains translation/permutation robustness but loses the spatial information that distinguishes the two patterns.

24.7 From Hand-Crafted to Learned Features

The previous chapter ended with a ladder whose top rungs — bag-of-keypoints and CNN features — both turn many *local* descriptions into one *global* vector, but differ in whether the building blocks are designed by hand or learned. This chapter walks that final stretch of the ladder: first the **Bag of Words** construction (the high point of hand-crafted methods), then the general **coding–pooling** pipeline that frames it, and finally the conceptual leap to **feature learning**.

24.7.1 Bag of Words: borrowing an idea from text analysis

The motivating problem (stated on the slide) is: *how do we build one global feature vector out of many local keypoints?* An image yields hundreds of local patches of *variable number and position*; a classifier needs a *single fixed-length* vector $\mathbf{x} \in \mathbb{R}^p$. Bag of Words is the bridge, and the inspiration comes from how documents are represented in text analysis.

The text analogy

In document analysis, a classic representation ignores word order and simply **counts how often each word occurs**. A text about Formula 1 will contain "Schumacher", "victory", "lap", "Grand Prix" many times and "screensaver" rarely — so the vector of word-counts already characterises the topic, *without any grammar or ordering*. This is the **bag of words**: a bag (multiset) keeps multiplicities but forgets order.

Bag of Words (text)

Fix a **vocabulary** of p words. Represent a document by the vector $\mathbf{x} \in \mathbb{R}^p$ whose j-th entry is the number of times word j appears. Word *order* is discarded; only *frequencies* remain. Documents on the same topic land near each other in $\mathbb{R}^p$.

Transferring it to images: Bag of Keypoints

Images have no natural "words", so we must *build* a visual vocabulary. The recipe (the slide calls the result a **Bag of Keypoints**):

1. **Extract local descriptors.** Detect many local patches across the image and describe each one with a local descriptor (e.g. SIFT). Each image now produces a *set* of descriptor vectors — variable in number, unordered.
2. **Build a visual vocabulary (the "codebook").** Pool descriptors from many training images and **cluster** them (typically k-means) into p groups. Each cluster centre is a **visual word** (a prototypical patch: a particular kind of corner, edge, texture). The set of p centres is the codebook.
3. **Quantise (code).** For a given image, assign each of its local descriptors to the nearest visual word.
4. **Build the histogram (pool).** Count how many descriptors fell into each visual word. The resulting vector of p counts is the image's Bag of Keypoints — a fixed-length $\mathbf{x} \in \mathbb{R}^p$, regardless of how many keypoints the image had.

Key idea

Why this solves the original problem. Clustering converts a continuous, variable-size set of local descriptors into counts over a *fixed* vocabulary of size p. "Variable number of unordered local patches" becomes "one fixed-length histogram" — exactly the $\mathbf{x} \in \mathbb{R}^p$ the classifier needs. Like the text version, it discards *where* the words occurred, which buys robustness to position, small deformations and clutter.

Numeric example: quantising five keypoints into a 3-word vocabulary

Suppose the codebook has $p = 3$ visual words $\{w_1, w_2, w_3\}$ and an image produces 5 local descriptors, assigned to their nearest words as

$$d_1 \to w_2, \quad d_2 \to w_1, \quad d_3 \to w_2, \quad d_4 \to w_3, \quad d_5 \to w_2.$$

Counting occurrences per word gives the Bag-of-Keypoints histogram

$$\mathbf{x} = (\underbrace{1}_{w_1}, \underbrace{3}_{w_2}, \underbrace{1}_{w_3}) \in \mathbb{R}^3.$$

A different image with 200 keypoints would still produce a length-3 vector — only the counts change. Often one **normalises** so the entries sum to 1 (here $(0.2, 0.6, 0.2)$), making the descriptor invariant to the total number of keypoints, i.e. to image size.

24.7.2 The coding–pooling pipeline

Bag of Words is one instance of a general two-stage template that the slide lays out as a full image-classification pipeline. Reading it left to right:

$$\text{Image} \rightarrow \boxed{\text{Local descriptors}} \rightarrow \boxed{\text{Coding}} \rightarrow \boxed{\text{Pooling}} \rightarrow \boxed{\text{Linear classifier}} \rightarrow \text{label}$$

Coding and pooling

Coding re-expresses each local descriptor with respect to the codebook. In plain Bag of Words this is hard assignment to the nearest visual word; richer variants (the slide names *VQ* — vector quantisation, *SC* — sparse coding, *LLC* — locality-constrained linear coding) assign each descriptor *softly* to several words with weights, losing less information.

Pooling aggregates all the codes of an image into one fixed-length vector. *Sum/average pooling* reproduces the histogram of counts; *max pooling* instead records, for each visual word, the strongest response anywhere in the image.

The slide separates the pipeline into two regimes:

- the descriptor–coding–pooling part is **unsupervised** — it uses no labels (the codebook comes from clustering, the descriptors are hand-designed). It is purely a way of *building the representation* $\mathbf{x}$.
- the final **classifier** is **supervised** — it is trained on the labels y_i. The slide names typical choices (*SVM*, *MRF*, *BN*); a *linear* classifier usually suffices once the representation is good.

Putting the location back: Spatial Pyramids

Pure Bag of Words throws away *all* spatial information — a face and its scrambled pieces give the same histogram. The fix shown on the slide ("pre-defined partitioning", **SPR** = Spatial Pyramid Representation) is to pool not only over the whole image but also over a grid of sub-regions, at several levels of resolution.

Key idea

Spatial pyramid. Partition the image at increasingly fine levels: level $\ell = 0$ is the whole image (1 cell); level $\ell = 1$ splits it into $2 \times 2 = 4$ cells; level $\ell = 2$ into $4 \times 4 = 16$ cells. Compute a Bag-of-Words histogram *within each cell* and concatenate them all. This reintroduces a *coarse* sense of *where* things are (top-left vs. bottom-right) while keeping the robustness of local histograms.

Numeric example: counting the 21 spatial bins

With levels $\ell = 0, 1, 2$ the number of cells is

$$\underbrace{1}_{\ell=0} + \underbrace{4}_{\ell=1} + \underbrace{16}_{\ell=2} = 21 \text{ cells},$$

which is the "21 Spatial Bins" annotation on the slide. If each cell holds a Bag-of-Words histogram over a vocabulary of size p, the full pyramid descriptor has length

$$21 \times p.$$

For example a vocabulary of $p = 200$ visual words yields a $21 \times 200 = 4200$-dimensional image descriptor. The price of restoring spatial information is this growth in dimensionality.

From feature engineering to feature learning

Everything so far — the keypoint detector, the descriptor, the pooling scheme, the pyramid — was **designed by hand**. The slide poses the question that ends the classical era:

Is it possible to learn the most appropriate representation, possibly with a hierarchy, directly from the data?

The answer is yes, and it reframes the whole enterprise as a shift **from feature engineering to feature learning**.

Engineering vs. learning

Feature engineering: a human decides the form of ϕ (which patches, which descriptor, which pooling). The data is used only to fit the final classifier.

Feature learning: the representation ϕ *itself* is learned from data, jointly with the classifier. The system discovers which features are useful rather than being told.

The slide also stresses that the learned representation is **hierarchical** — organised into levels of increasing abstraction:

- **Low-level features:** tiny, generic patterns — oriented edges, spots, simple gradients. They look like the local descriptors we used to design by hand, except here they are *learned*.
- **Mid-level features:** combinations of low-level ones into parts — corners, curves, textured fragments, simple object parts.
- **High-level features:** combinations of parts into object-like structures — e.g. whole faces emerging in the visualisations on the slide.

Key idea

The punchline. A hierarchy that builds edges → parts → objects, *learned end-to-end from data*, is exactly what a Convolutional Neural Network provides. The hand-crafted coding–pooling pipeline and the learned CNN hierarchy are doing the *same job* (turn local evidence into a global, robust, discriminative representation) — but the CNN *learns* every stage instead of having it specified by an engineer. This is the doorway to the next chapter.

24.8 Convolutional Neural Networks

The previous chapter closed on a promise: a representation that builds edges → parts → objects, *learned* end-to-end from data, is exactly what a **Convolutional Neural Network** (CNN) delivers. This chapter unpacks the typical CNN, the softmax that turns its outputs into class probabilities, the dataset (ImageNet) that made it work, what the learned features look like, and two practical pillars — transfer learning and data augmentation.

24.8.1 A typical CNN: two halves

The slide draws a CNN as a left-to-right stack that splits cleanly into two functional halves:

$$\underbrace{\text{Conv} + \text{ReLU} \to \text{Pool} \to \text{Conv} + \text{ReLU} \to \text{Pool} \to \cdots}_{\textbf{feature learning}} \quad \to \quad \underbrace{\text{Flatten} \to \text{Fully connected} \to \text{Softmax}}_{\textbf{classification}}$$

Key idea

A CNN is the whole Section 24.7 pipeline, learned. The feature-learning half plays the role of ϕ (descriptors + coding + pooling), and the classification half plays the role of the supervised classifier — but now *every* stage is trained jointly from the labels. There is no hand-designed codebook; the network discovers its own features.

The building blocks

The layers of the feature-learning half

Convolution: slide a small learnable filter (kernel) across the image, computing a local weighted sum at every position. One filter produces one *feature map* highlighting where its pattern occurs. Crucially the same weights are reused at every location (*weight sharing*), so the layer is **translation-equivariant** and uses few parameters.

ReLU (Rectified Linear Unit): the nonlinearity $\mathrm{ReLU}(z) = \max(0, z)$, applied elementwise. Without a nonlinearity, stacking layers would collapse into a single linear map; ReLU lets the network represent complex functions, and is cheap (zero out negatives, keep positives).

Pooling: downsample each feature map, typically by *max pooling* over small windows (keep the strongest response). This shrinks spatial resolution, grants robustness to small translations, and enlarges the *receptive field* of later layers — echoing exactly the pooling stage of Section 24.7.

The repetition conv → ReLU → pool is what produces the **hierarchy** of previus Sections: early layers see tiny patches and learn low-level edges; after pooling, later layers see larger regions and combine edges into parts, then parts into objects.

The classification half

Flatten: unroll the final stack of feature maps into one long vector — this vector *is* the learned representation $\mathbf{x}$.

Fully connected (dense) layers: ordinary linear layers (every input connected to every output) that combine the features into K scores, one per class. These scores are called **logits**.

Softmax: converts the K logits into a probability distribution over the classes (next section).

Numeric example: a 3×3 convolution at one position

Take a 3×3 image patch and a 3×3 filter:

$$P = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix}, \qquad W = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

The convolution output at this location is the elementwise product summed:

$$\sum_{u,v} P_{uv} W_{uv} = (1)(1) + (0)(0) + (2)(-1) + (0)(1) + \cdots + (1)(-1) = -2.$$

Applying ReLU: $\max(0, -2) = 0$. This particular W responds to a left-to-right intensity *drop* (a vertical edge); here the patch has no such edge, so after ReLU the response is zero. Sliding W over the whole image produces one feature map of such responses.

24.8.2 The softmax function

The final layer outputs K real-valued logits $y_1, \ldots, y_K$ (one per class), which can be any size, positive or negative. To interpret them as **class probabilities** we apply the softmax (the formula on the slide):

$$\mathrm{softmax}(y_i) = \frac{e^{y_i}}{\sum_{j=1}^{K} e^{y_j}}.$$

Why this particular formula

- The exponential e^{y_i} is always positive, so no probability comes out negative.
- Dividing by the sum $\sum_j e^{y_j}$ **normalises** the outputs so they add up to 1 — a valid probability distribution.
- Exponentiation is *monotonic*, so the largest logit stays the largest probability; softmax exaggerates differences (the biggest logit gets a disproportionately large share).

The predicted class is then $\hat{y} = \arg\max_i \text{softmax}(y_i)$, which is just the index of the largest logit.

Numeric example: softmax over three classes

Suppose the network outputs logits $y = (2.0,\ 1.0,\ 0.1)$ for classes (car, truck, van). Exponentiate:

$$e^{2.0} \approx 7.389, \quad e^{1.0} \approx 2.718, \quad e^{0.1} \approx 1.105.$$

Their sum is $7.389 + 2.718 + 1.105 = 11.212$. Dividing each term:

$$\text{softmax}(y) \approx \left(\tfrac{7.389}{11.212},\ \tfrac{2.718}{11.212},\ \tfrac{1.105}{11.212}\right) \approx (0.659,\ 0.242,\ 0.099).$$

The probabilities sum to 1, and the model predicts `car` with $\approx 66\%$ confidence. Note how a logit gap of 2.0 vs 1.0 became a probability gap of 0.66 vs 0.24 — the exponential sharpens the contrast.

24.8.3 The key to CNN success: ImageNet

CNNs were known long before they dominated; what changed around 2012 was the arrival of a large enough labelled dataset. The slide credits **ImageNet** (Deng et al., CVPR 2009).

- $\sim$**14 million** labelled images across $\sim$**20,000** classes, gathered from the Internet.
- Labels obtained by **crowdsourcing** (Amazon Mechanical Turk) — manual annotation at massive scale.
- The associated **ImageNet Challenge (ILSVRC, 2012)** used a subset of $\sim$**1.2 million** training images over **1000** classes.

Key idea

Data was the missing ingredient. Large CNNs have millions of parameters; fitting them without overfitting requires correspondingly huge labelled datasets. ImageNet supplied that scale, and the dramatic CNN win in the 2012 challenge is what launched the deep-learning era in vision. This is why the slide calls ImageNet "the key of success".

24.8.4 Visualising what CNNs learn

Because the features are learned rather than designed, it is natural to ask *what* the network actually represents. The slide lists several complementary techniques (pointer: `cs231n.github.io/understanding-cnn/`).

Four ways to look inside a CNN

Layer activations: feed an image and display each feature map — which neurons fire where. Early layers light up on edges; deeper layers on object parts.

Learnt filters: directly visualise the convolution kernels. First-layer filters typically look like oriented edge and colour detectors (Gabor-like), confirming the low-level rung of the hierarchy.

Images maximising a class score: synthesise, by optimisation, the input that most strongly activates a given class — revealing what the network "thinks" that class looks like (often a dreamy texture of repeated parts).

Class saliency maps: for a *specific* input, highlight which pixels most influence the class score — showing *where* the network is looking (e.g. the sail of a sailboat).

These visualisations empirically confirm the edges → parts → objects hierarchy promised in Chapter 4.

24.8.5 Transfer learning and pre-trained features

Training a large CNN from scratch needs ImageNet-scale data, which most real problems lack. **Transfer learning** reuses the knowledge already captured in a model trained on ImageNet, applying it to a new task or dataset (within some limits).

Two ways to transfer

CNN as a feature extractor: take a pre-trained network, chop off its classification head, and use the activations of an inner layer as a ready-made representation $\mathbf{x}$. Then train only a fresh, small classifier on top. The expensive feature-learning half is reused *as is*.

Fine-tuning: start from the pre-trained weights and continue backpropagation on the new data, adapting *some or all* of the weights to the new problem. Typically the early (generic edge/texture) layers are kept frozen or changed little, while the later (task-specific) layers are adapted more.

Key idea

Why it works. Low-level features (edges, textures) are *generic* — useful for almost any vision task — so they transfer well. Only the higher, task-specific layers need adapting. Many models pre-trained on ImageNet are freely available, making transfer learning the default starting point when data is limited.

24.8.6 Coping with limited data: data augmentation

Even with transfer learning, "large architectures are very data hungry". A cheap and powerful remedy is **data augmentation**: manufacture extra training examples by applying *label-preserving* transformations to the images you already have.

- Typical transformations: **rotations**, **scaling**/cropping, flips, changes in **lighting**/colour, small translations.
- The key constraint is that the transformation must *not change the label* — a rotated dog is still a dog. Augmentation effectively teaches the network the invariances of Chapter 3 (to viewpoint, scale, illumination) by *showing* it varied examples rather than building invariance in by hand.

The slide also mentions a more recent strategy: training on **synthetic data** (computer-generated images). This can supply unlimited labelled data, but a **semantic gap** remains between synthetic and real images, so *fine-tuning on real data* is usually still needed to close it.

Numeric example: how augmentation multiplies a dataset

Start with 1000 labelled images. Apply a small set of label-preserving transforms: 4 rotations $\times$ 2 horizontal flips $\times$ 3 brightness levels gives

$$4 \times 2 \times 3 = 24$$

variants per image, so the effective training set grows to $1000 \times 24 = 24{,}000$ images — at essentially zero labelling cost. The network sees the same objects under many nuisance variations, directly encouraging the robustness that Chapter 3 demanded of ϕ.

Chapter 25

Object Detection

So far every task has produced *one* answer for the whole image. Detection breaks that assumption: an image may contain *several* objects, at unknown positions and sizes, and we must find them *all*. This chapter formalises the task, derives the classic sliding-window solution from the classification machinery we already have, explains why it is expensive (especially with CNNs), surveys modern detector families, and finally places detection on a spectrum between plain classification and pixel-level segmentation.

25.1 Detection as classification of regions

The slide states the core insight plainly: *object detection is in essence a classification problem.* The twist is *what* we classify — not the whole image, but **image regions of variable size**, each tested with the question "is it an instance of the object, or not?".

Key idea

Detection = classification + localisation. Instead of one label for the image, we slide a window over many candidate regions and classify each as object / background. The positions of the windows that say "object" give us the *locations*; the classifier gives us the *labels*. Every tool from Chapters 1–5 carries over — we just apply it to regions.

25.1.1 The unbalanced-class problem

A special but very common case is having **one class of interest** (faces, pedestrians). The slide flags a statistical difficulty: the classes are wildly **unbalanced**. In a 380×220 image one may test on the order of 6.5×10^5 candidate windows, yet only a handful (e.g. 11 faces) are positive.

Training set for 1-class detection

The training data contains

- **positive examples:** image patches *containing* the object;
- **negative examples:** "background" patches *not* containing it.

Because background is everywhere and objects are rare, negatives vastly outnumber positives — the source of the class imbalance.

Numeric example: how extreme the imbalance is

With $\sim 6.5 \times 10^5$ tested windows and only 11 true positives, the fraction of positive windows is

$$\frac{11}{6.5 \times 10^5} \approx 1.7 \times 10^{-5} \approx 0.0017\%.$$

A lazy detector that simply answers "background" to *every* window would be correct on

$$\frac{6.5 \times 10^5 - 11}{6.5 \times 10^5} \approx 99.998\%$$

of windows — yet it finds *zero* faces. This is why raw accuracy is useless for detection and why we need the metrics of the next chapter (precision, recall), and why training must deliberately handle the imbalance.

25.2 Formalising the task

25.2.1 Single-class object detection

Given a class of interest (Face, Pedestrian, ...) and an input image I containing N instances, **locate all instances** — that is, find

$$\{(x_i,\, y_i,\, w_i,\, h_i)\}_{i=1}^{N},$$

where each tuple is a **bounding box**: (x_i, y_i) its centre (or top-left corner) and (w_i, h_i) its width and height. The output is a *set* of boxes, one per detected object.

25.2.2 Multi-class object detection

Given a *set* of classes of interest C (e.g. street objects: car, truck, person, traffic light) and an image I with N instances, find for each the box *and* its class label:

$$\{(c_i,\, x_i,\, y_i,\, w_i,\, h_i)\}_{i=1}^{N}, \qquad c_i \in C.$$

Key idea

The only change from single- to multi-class is the extra component $c_i \in C$ in each tuple: every detection now carries *which* class it is, not just *where* it is. The boxes are taken axis-aligned (aligned with the image coordinate system), which is the standard convention.

25.3 The sliding-window approach

How do we generate the candidate regions? The classic answer: **slide a window across the image and evaluate an object model at every location** — and "evaluate an object model" means *perform image classification* on the window. Ask at each position: *is it a face? yes / no.*

To handle objects of different sizes, the window is applied at multiple **scales** (equivalently, the image is resized and re-scanned). So the search runs over all positions *and* all scales.

25.3.1 Non-maximum suppression (NMS)

A single object usually triggers *many* overlapping positive windows (slightly shifted or rescaled boxes all say "face"). We want one box per object, so we apply **non-maximum suppression**.

Non-maximum suppression

Among a cluster of overlapping detections of the same object, **keep the one with the highest confidence score and suppress (delete) the others** that overlap it heavily. Repeat until no overlapping duplicates remain. The result is a single clean box per object instead of a messy pile of near-duplicates.

Numeric example: NMS on three overlapping boxes

Suppose a face fires three detections with confidence scores

$$b_1 : 0.95, \qquad b_2 : 0.80, \qquad b_3 : 0.62,$$

all heavily overlapping. NMS sorts by score, keeps b_1 (the maximum), then discards b_2 and b_3 because they overlap b_1 too much. One object $\Rightarrow$ one final box (b_1). (How much overlap counts as "too much" is measured by the IoU threshold of the next chapter.)

25.4 Sliding windows with CNNs

Can we just plug a CNN in as the per-window classifier? Yes in principle: apply the CNN to many crops of the image, and let it classify each crop as object/background (or as one of the classes). The slide walks through crops returning `Dog? no / Cat? no / Background? yes`, then a crop on the dog returning `Dog? yes`, and so on.

Key idea

The cost problem. A CNN is expensive to evaluate *once*; the sliding window asks us to evaluate it on a *huge* number of locations *and* scales. Naively, "apply a CNN to every crop at every scale" is **computationally prohibitive**. This bottleneck is precisely what modern detector architectures were invented to avoid.

Numeric example: why brute-force crops explode

Imagine evaluating windows at every pixel of a 380×220 image, across, say, 5 scales:

$$380 \times 220 \times 5 \approx 4.2 \times 10^5 \text{ CNN evaluations per image.}$$

If one CNN forward pass takes 5 ms, a single image would need

$$4.2 \times 10^5 \times 5\,\text{ms} \approx 2100\,\text{s} \approx 35\,\text{minutes.}$$

Clearly unusable for real-time detection — hence the need for smarter designs that avoid re-running the network on every crop.

25.5 Modern detector families

The slide lists the main supervised approaches (it deliberately restricts to supervised methods).

Three families of detectors

Two-stage CNN detectors — e.g. the **R-CNN family** (R-CNN, Fast R-CNN, Faster R-CNN). Stage 1 proposes a manageable set of candidate regions (instead of scanning everything); stage 2 classifies and refines each proposal. Accurate, historically a bit slower.

One-stage CNN detectors — e.g. the **YOLO family** ("You Only Look Once"). Predict boxes and classes *directly* in a single network pass over the image, with no separate proposal stage. Much faster, well suited to real-time use.

Transformer-based methods — e.g. **DETR** (DEtection TRansformer). Cast detection as a set-prediction problem solved with attention, removing hand-designed components such as anchor boxes and NMS.

These all share the goal of *avoiding* the brute-force "CNN on every crop" cost: two-stage methods cut the number of regions; one-stage methods cut the number of passes to one.

25.6 Different nuances: a spectrum of tasks

The slide arranges four related tasks on a spectrum from coarse to fine, splitting them into *single-object* and *multiple-object* regimes.

Four levels of image understanding

Classification *(single object):* one label for the image ("CAT"). *What* is here.

Classification + localisation *(single object):* one label plus one bounding box ("CAT" here). *What* and *where*, for a single object.

Object detection *(multiple objects):* a label and a box for *each* of several objects ("CAT, DOG, DUCK"). *What* and *where*, repeated.

Instance segmentation *(multiple objects):* a label and a precise *pixel mask* (not just a box) for each object. *What*, *where*, and the exact *shape*.

25.6.1 Detection vs. semantic segmentation

The slide also contrasts detection with **semantic segmentation**, and the difference is worth stating sharply.

Key idea

Semantic segmentation labels every pixel. Its goal is to associate a semantic attribute (Sky, Grass, Cat, Cow, Trees) to *every single pixel* of the image — a dense, pixel-wise classification. Detection instead outputs a few *boxes*. Note the consequence shown on the slide: plain semantic segmentation marks all "cow" pixels as `cow` but does *not* separate two overlapping cows — it gives the *class* of each pixel, not the *instance*. Telling individual objects apart is the job of *instance* segmentation.

Numeric example: what each task outputs for one image

For a photo containing a cat, a dog and a duck:

- **Classification:** one label, e.g. `cat` (the dominant object) — a single value.
- **Detection:** 3 tuples, e.g. $(\texttt{cat}, x_1, y_1, w_1, h_1)$, $(\texttt{dog}, \dots)$, $(\texttt{duck}, \dots)$ — 3 boxes.
- **Semantic segmentation:** for a 100×100 image, a label for each of the $100 \times 100 = 10{,}000$ pixels — a full label *map*, but two adjacent dogs would share the label `dog` with no boundary between them.
- **Instance segmentation:** the same 10,000-pixel map, but with each object's pixels tagged by *instance* (dog#1 vs dog#2).

The output grows from a single number, to a few boxes, to a dense per-pixel map — increasing spatial precision at increasing cost.

Chapter 26

Object Detection Evaluation

We built detectors in Chapter 25; now we must *measure* them. The previous chapter already warned that raw accuracy is meaningless under extreme class imbalance (a detector saying "background" everywhere scores 99.998% yet finds nothing). This final chapter develops the metrics that actually work: **IoU** to decide when a box is correct, the **TP/FP/FN/TN** bookkeeping, and the summary scores — **ROC**, **precision / recall / F1**, the **precision–recall curve** with **AP/mAP**, and **confusion matrices**.

26.1 Intersection over Union (IoU)

A predicted box is almost never *pixel-perfect*, so we need a graded measure of how well it matches the ground-truth box. That measure is the **Intersection over Union**, also called the **Jaccard index**.

IoU / Jaccard metric

For a predicted box P and a ground-truth box G,

$$\mathrm{IoU}(P, G) = \frac{\mathrm{area}(P \cap G)}{\mathrm{area}(P \cup G)} = \frac{\text{area of overlap}}{\text{area of union}} \in [0, 1].$$

IoU $= 1$ means the boxes coincide exactly; IoU $= 0$ means they do not overlap at all. It rewards both *correct position* and *correct size* in a single number.

Numeric example: computing an IoU

Let the ground truth be a 10×10 box and the prediction a 10×10 box shifted so that they overlap in a 6×8 rectangle. Then

$$\mathrm{area}(P \cap G) = 6 \times 8 = 48,$$

$$\mathrm{area}(P \cup G) = \underbrace{100}_{G} + \underbrace{100}_{P} - \underbrace{48}_{\cap} = 152,$$

$$\mathrm{IoU} = \frac{48}{152} \approx 0.316.$$

With the usual threshold of 0.5 (below), this prediction would *not* count as a correct detection — it overlaps too little.

26.2 From IoU to TP / FP / FN / TN

IoU gives a number; to count successes and failures we threshold it. Pick a threshold (rule of thumb: IoU ≥ 0.5). Then each prediction / ground-truth pairing falls into one of four categories.

The four outcomes (binary detection)

True Positive (TP): a prediction matches a ground-truth box with IoU $\geq$ threshold. A correct detection.

False Positive (FP): a prediction with IoU $<$ threshold (it does not match any real object well enough) — a spurious detection, including the "low overlap" case where a box lands near an object but not tightly enough.

False Negative (FN): a ground-truth object with *no* associated prediction — a missed object.

True Negative (TN): all the other regions (correctly *not* detected). In detection these are essentially uncountable — almost the entire image is "correctly empty" — which is exactly why TN-based measures like accuracy are avoided here.

Key idea

Read the slide's examples through these definitions. A green box hugging the face (high IoU) is a **TP**; a green box floating on the background is a **FP**; a red ground-truth face with no green box on it is a **FN**; a green box that grazes the face with too little overlap is simultaneously a **FP** (the bad prediction) and leaves a **FN** (the object still unmatched). Position *and* tightness both matter.

26.3 The ROC curve

The **Receiver Operating Characteristic** curve illustrates the performance of a binary classifier as its decision threshold varies. It plots

$$\text{True Positive Rate (TPR, sensitivity)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{against} \quad \text{False Positive Rate (FPR)} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

Key idea

Reading ROC space. The top-left corner ($\text{FPR} = 0, \text{TPR} = 1$) is the *perfect* classifier — all objects found, no false alarms. The diagonal is *random guessing*. A point *above* the diagonal is "better", *below* it is "worse". A better classifier bows toward the top-left; the curve's shape lets you pick the threshold trading off misses against false alarms.

Note the dependence on TN through the FPR. Because detection has essentially unlimited TNs, the ROC curve is more natural for balanced classification than for detection — which is why detection leans on precision–recall instead.

26.4 Precision, Recall and the F1 score

These two metrics avoid TN entirely, making them the workhorses of detection.

Precision and recall

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{\text{TP}}{\text{estimated positives}},$$

of the things I detected, what fraction were real? (penalises false alarms).

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{real positives}},$$

of the real objects, what fraction did I find? (penalises misses).

The two pull against each other: lowering the confidence threshold detects more objects (recall ↑) but also more junk (precision ↓). The **F1 score** combines them into one number via their *harmonic mean*:

$$F_1 = 2\,\frac{\text{precision}\cdot\text{recall}}{\text{precision}+\text{recall}} = \frac{2\,\text{TP}}{2\,\text{TP}+\text{FP}+\text{FN}}.$$

Key idea

Why the harmonic mean. F_1 is high only when *both* precision and recall are high; if either is near zero, F_1 is dragged down. The second form, $\frac{2TP}{2TP+FP+FN}$, shows it cleanly ignores TN — ideal for the imbalanced detection setting.

Numeric example: precision, recall and F1 together

A detector returns 8 boxes; 6 are correct (TP) and 2 are false alarms (FP). The image actually contained 10 objects, so 4 were missed (FN).

$$\text{Precision} = \frac{6}{6+2} = 0.75, \qquad \text{Recall} = \frac{6}{6+4} = 0.60.$$

$$F_1 = 2\cdot\frac{0.75\times 0.60}{0.75+0.60} = 2\cdot\frac{0.45}{1.35} \approx 0.667.$$

Cross-check with the TP form: $\frac{2\cdot 6}{2\cdot 6+2+4} = \frac{12}{18} \approx 0.667$. The two formulas agree.

26.5 The precision–recall curve, AP and mAP

A detector assigns each box a **confidence score**. Sweeping a threshold over that score traces out a whole curve of (recall, precision) operating points — the **precision–recall (P–R) curve**.

Key idea

Reading the P–R curve. At a *high* confidence threshold (e.g. 0.9) the detector reports only its surest boxes: precision is high, recall is low (e.g. $P = 0.98,\ R = 0.3$). At a *low* threshold (e.g. 0.1) it reports almost everything: recall is high, precision is low (e.g. $P = 0.2,\ R = 0.9$). The "ideal" corner is top-right — high precision *and* high recall simultaneously.

Average Precision (AP) and mean AP (mAP)

AP = the *area under the precision–recall curve* for one class. A single number in $[0, 1]$ summarising the whole precision/recall trade-off; closer to 1 is better.

mAP = the *mean of AP across all classes*. The standard single-number score for multi-class detectors.

Numeric example: mAP from per-class AP

A 3-class detector achieves

$$\text{AP}_{\text{car}} = 0.80, \quad \text{AP}_{\text{person}} = 0.65, \quad \text{AP}_{\text{bicycle}} = 0.50.$$

Then

$$\text{mAP} = \frac{0.80+0.65+0.50}{3} = \frac{1.95}{3} = 0.65.$$

The single number 0.65 summarises the detector's quality across all three classes.

26.6 Confusion matrices: multi-class evaluation

For multi-class problems, a **confusion matrix** C shows *how* the errors are distributed, not just how many. Rows index the **ground-truth** class, columns the **predicted** class; entry $C(i, j)$ counts examples of true class i predicted as class j.

Reading the confusion matrix

Diagonal $C(i, i)$: correct predictions (true positives for class i).

Off-diagonal, row i: the true-i examples sent to other classes — the **false negatives** of class i (it was an i, called something else).

Off-diagonal, column j: examples of other classes predicted as j — the **false positives** of class j (called a j, wasn't one).

A good classifier has mass concentrated on the diagonal. Large off-diagonal entries reveal *which* pairs of classes get confused with each other.

The **overall accuracy** is the total correct over the total count — the sum of the diagonal:

$$\mathrm{Acc}_{\mathrm{overall}} = \frac{\sum_i C(i,i)}{\sum_{i,j} C(i,j)}.$$

(The slide writes the numerator $\sum_i C(i, i)$, the count of all correctly classified examples; dividing by the grand total turns it into a rate.)

Numeric example: a 3×3 confusion matrix

Three classes, with ground truth in rows and predictions in columns:

$$C = \begin{array}{c|ccc} & \hat{C}_1 & \hat{C}_2 & \hat{C}_3 \\ \hline C_1 & 20 & 3 & 2 \\ C_2 & 4 & 18 & 1 \\ C_3 & 0 & 5 & 25 \end{array}$$

Diagonal (correct): $20 + 18 + 25 = 63$. Grand total: sum of all 9 entries $= 78$. Overall accuracy

$$\mathrm{Acc}_{\mathrm{overall}} = \frac{63}{78} \approx 0.808.$$

Per-class reading of C_1: recall $= \frac{20}{20+3+2} = \frac{20}{25} = 0.80$ (of true C_1, 80% found); precision $= \frac{20}{20+4+0} = \frac{20}{24} \approx 0.83$ (of those *called* C_1, 83% were right). The off-diagonal $C(3, 2) = 5$ shows class C_3 is most often confused with C_2.

Closing remark

These metrics close the loop opened in Chapter 24. We began by turning images into vectors $\mathbf{x}$ and labels y; built representations (hand-crafted, then learned); used CNNs to classify, detect and segment; and now we can *quantify* how well any of it works — with measures (IoU, precision, recall, F1, AP/mAP, confusion matrices) designed to stay honest even under the severe class imbalance that real detection entails.

Chapter 27

The R-CNN Family and Instance Segmentation

Chapter 25 ended on a cliff-hanger: applying a CNN to every sliding-window crop at every scale is *computationally prohibitive*, and modern detectors were invented to avoid exactly this cost. This chapter develops that line of work — the **R-CNN family** — from the single-object case up to pixel-accurate instance segmentation with Mask R-CNN. The recurring theme is a sequence of engineering fixes, each removing the bottleneck left by the previous design.

27.1 Classification with localization: a multitask network

Before tackling *many* objects, consider the easier single-object task from the spectrum of Chapter 25: **classification + localization**. We want one label *and* one bounding box for the dominant object.

The architecture reuses a CNN backbone exactly as in Chapter 24: the image is passed through convolution/pooling layers and flattened into a feature vector (the slides use a 4096-dimensional vector, the classic VGG/AlexNet size). From that single shared vector, **two heads** branch off:

The two output heads

Classification head: a fully connected layer mapping the 4096-vector to K class scores (the slides show $4096 \to 1000$ for the 1000 ImageNet classes), turned into probabilities by softmax (Chapter 24).

Localization head: a fully connected layer mapping the *same* 4096-vector to 4 numbers — the box coordinates (x, y, w, h) (so $4096 \to 4$).

Key idea

Localization is treated as regression. The classification head predicts a discrete label (trained with a *softmax / cross-entropy loss* against the correct label). The localization head predicts 4 continuous numbers, so it is a **regression** problem, trained with an **L2 loss** against the correct box (x', y', w', h'):

$$L_{\text{box}} = \|(x, y, w, h) - (x', y', w', h')\|_2^2.$$

This is the first time a head outputs real-valued coordinates rather than a class — a pattern that recurs everywhere below.

27.1.1 The multitask loss

The network is trained to do *both* jobs at once by summing the two losses into a single **multitask loss**:

$$L_{\text{total}} = L_{\text{cls}} + \lambda\, L_{\text{box}},$$

where $\lambda > 0$ balances the two terms (classification in "probability" units, regression in "pixel" units, so they need rescaling). A single backward pass then trains the shared backbone and both heads jointly.

Numeric example: why a balancing weight λ is needed

Suppose at some training step the classification loss is $L_{\text{cls}} = 0.7$ (a typical cross-entropy value) while the box coordinates are in pixels and the L2 error is $L_{\text{box}} = (12)^2 + (8)^2 + (5)^2 + (5)^2 = 258$. Without weighting,

$$L_{\text{total}} = 0.7 + 258 \approx 258.7,$$

so the gradient is almost entirely driven by the box term — the classifier would barely learn. Choosing e.g. $\lambda = 0.002$ gives

$$L_{\text{total}} = 0.7 + 0.002 \times 258 \approx 1.22,$$

putting the two tasks on a comparable footing. The exact λ is a hyperparameter tuned on validation data.

This single-object design does not yet handle *several* objects — we cannot output a fixed 4 numbers when the number of objects is unknown. That is what region proposals solve.

27.2 Region proposals and selective search

The brute-force sliding window tested *every* location and scale. The key idea that breaks the cost barrier is to test only a *small, promising* subset of regions.

Region proposals

A **region proposal** algorithm scans the image and returns a manageable set of candidate boxes that are *likely to contain some object* — regardless of which class. Instead of $\sim 10^5$ windows we get $\sim$2000 proposals, which a CNN can afford to examine.

The classic proposal method named on the slides is **selective search** (Uijlings et al., 2013). It over-segments the image into small regions (by colour/texture similarity) and *greedily merges* adjacent regions into larger ones, emitting boxes at every level of merging — so it proposes objects at many scales.

Key idea

High recall, low precision — on purpose. Selective search is designed to *rarely miss* a real object (high recall) at the cost of producing *many* junk boxes (low precision). The reasoning: a later CNN classifier can always *reject* a false proposal, but it can never *recover* an object that was never proposed. So it is safe to over-propose. (Recall and precision are exactly the metrics from the previous evaluation chapter.)

27.3 R-CNN: regions with CNN features

The first member of the family, **R-CNN** (Girshick et al.), combines the two ideas above into a two-step pipeline.

R-CNN pipeline

1. **Propose:** run selective search to get $\sim$2000 Regions of Interest (RoIs).
2. **Warp:** crop each RoI and *resize* it to the fixed input size the CNN expects (the slides note 224×224).
3. **Extract:** forward each warped region *independently* through the CNN to compute its feature vector.
4. **Classify & refine:** classify each region (the slides note an external classifier such as an **SVM** may be used per class) *and* run a **bounding-box regression** predicting 4 corrections (dx, dy, dw, dh) that nudge the proposal toward the true box.

Key idea

The R-CNN bottleneck. Step 3 runs the CNN *separately* on each of the ~ 2000 regions per image — $\sim$2000 independent forward passes. This is "accurate but slow". The fix, which defines the next model, is to notice that these regions overlap heavily and re-compute the same convolutional features over and over.

Numeric example: counting R-CNN forward passes

With 2000 proposals per image and a CNN forward pass of, say, 10 ms:

$$2000 \times 10\,\text{ms} = 20{,}000\,\text{ms} = 20\,\text{s per image.}$$

Far too slow for practical use — and almost all of that work is redundant, because overlapping crops share most of their pixels (and hence their features).

27.4 Fast R-CNN: share the convolution

Fast R-CNN reorders the operations to kill the redundancy: *"pass the image through the convnet* before *cropping, and crop the* feature map *instead of the image."*

Fast R-CNN pipeline

1. Run the CNN *once* on the *whole* image, producing a single convolutional feature map.
2. Still obtain proposals from selective search, but *project* each proposal onto the shared feature map.
3. For each projected RoI, apply **RoI pooling** (below) to get a fixed-size feature, then pass it to the FC layers and the two heads (class + box).

Key idea

The decisive change. The expensive convolutions are computed *once per image* instead of once per region. The $\sim$2000 regions now share the same feature map, so per-region cost is just the cheap RoI pooling + small FC heads — a huge speed-up over R-CNN's 2000 full passes.

27.4.1 RoI pooling

The FC heads need a *fixed-size* input (e.g. 7×7), but proposals come in arbitrary sizes. **RoI pooling** solves this.

RoI pooling

Project the RoI onto the feature map, then *divide it into a fixed* 7×7 *grid* of (roughly) equal cells and max-pool within each cell. Any input region, large or small, thus produces the same 7×7 output — the fixed-length representation the heads require.

Numeric example: the two quantizations in RoI pooling

The slides work an example with a VGG16 backbone that downsamples by a factor of 32, on an 800×800 image, with a 665×665 object box.

- Mapping the image to the feature map: $800/32 = 25$ (a clean integer).
- Mapping the box: $665/32 = 20.78$ — *not* an integer. RoI pooling **rounds** it (to 20). This is the *first* quantization.
- Splitting that 20-wide region into a 7×7 grid: $20/7 = 2.86$ cells wide — again not an integer, so each cell boundary is *rounded again*. This is the *second* quantization.

Each rounding shifts the pooled region by up to a fraction of a feature-map cell, which on the original image is up to ~ 32 pixels of misalignment. For coarse classification this is tolerable; for pixel-accurate masks it is not — a problem RoI Align fixes later.

27.5 Faster R-CNN: learn the proposals too

Fast R-CNN still calls *selective search*, a fixed, hand-designed, relatively slow module that is now the bottleneck. **Faster R-CNN** replaces it with a learned **Region Proposal Network (RPN)** that predicts proposals *from the same feature map*, so proposals become a learned, fast, GPU-friendly step.

27.5.1 The Region Proposal Network and anchors

Anchor boxes

At *each point* of the convolutional feature map, place one (or several) reference boxes of fixed size and shape, called **anchors**. An anchor is a *prior guess* at where an object might be, centred on that feature-map location. The RPN's job is to judge and adjust these anchors.

A small convolutional head slides over the feature map and, at every location, predicts two things per anchor:

- **Objectness** — a *binary* classification: does this anchor contain an object or background? (output size $1 \times 20 \times 15$ for one anchor over a 20×15 feature map in the slides' example).
- **Box corrections** — a *regression* of 4 numbers per anchor, nudging the anchor toward the true object box (output $4 \times 20 \times 15$). Only computed for the positive anchors.

In practice K anchors of different sizes/aspect ratios are used at each location, so the outputs become $K \times 20 \times 15$ (objectness) and $4K \times 20 \times 15$ (corrections). The anchors are finally **sorted by objectness score** and the top ~ 300 are kept as proposals.

Numeric example: how many anchors does the RPN score?

With a 20×15 feature map and K anchors per location, the total number of candidate boxes scored is

$$K \times 20 \times 15 = 300K.$$

For $K = 9$ (a common choice: 3 scales $\times$ 3 aspect ratios) that is 2700 anchors, from which the top ~ 300 by objectness are passed on. Compare this to brute-force sliding windows ($\sim 10^5$) — the RPN slashes the candidate count by orders of magnitude while *learning* which boxes matter.

Key idea

Faster R-CNN is trained with four losses jointly. (1) RPN objectness (object vs. not), (2) RPN box regression, (3) final classification over the object classes, (4) final box regression. The whole detector — backbone, RPN, heads — is now a *single end-to-end network* with no external proposal module. Note the NMS of Chapter 6 reappears inside the RPN to remove duplicate overlapping proposals.

27.5.2 The family in one line each

Model	What changed	Bottleneck removed
R-CNN	CNN features on ∼2000 warped crops + SVM + box regression	(baseline: slow)
Fast R-CNN	CNN run once on whole image; crop the *feature map* via RoI pooling	2000 redundant passes
Faster R-CNN	Learned RPN with anchors replaces selective search	slow external proposals

27.6 Mask R-CNN: instance segmentation

Mask R-CNN (He et al., 2017) extends Faster R-CNN from boxes to **instance segmentation** — the finest level of the Chapter 6 spectrum, where each object instance gets a pixel-accurate *mask*, and (unlike semantic segmentation) two overlapping objects of the same class are kept separate.

Mask R-CNN's third head

Take Faster R-CNN and add a *third* output head alongside class and box: a **mask head**, implemented as a small **fully convolutional network** that outputs a binary mask (object pixels vs. not) *for each RoI*. So each detection now carries: a class, a box, *and* a per-pixel mask.

27.6.1 RoI Align: removing the quantization

A pixel-accurate mask cannot tolerate the ∼32-pixel misalignment that RoI *pooling* introduced through its two roundings. Mask R-CNN therefore replaces RoI pooling with **RoI Align**, whose defining property is **no quantization**.

RoI Align

RoI Align keeps the projected RoI coordinates as *exact floating-point values* (no rounding of the region, and no rounding of the grid-cell boundaries). To read a value at a non-integer location of the feature map, it uses **bilinear interpolation** of the four nearest feature-map cells.

The slides walk a concrete example. The RoI maps to a region 6.25 cells wide and 4.53 tall (kept as decimals, *not* rounded). With a 3×3 output, the region is divided into 9 boxes of equal real-valued size; inside each box, 4 sample points are placed at fractional coordinates.

Numeric example: locating a sample point with no rounding

For a sub-box, sample-point coordinates are found by dividing the box width and height by 3 and stepping. The slides' first (top-left) point:

$$X = X_{\text{box}} + \tfrac{\text{width}}{3} \times 1 = 9.94, \qquad Y = Y_{\text{box}} + \tfrac{\text{height}}{3} \times 1 = 6.50,$$

giving the point (9.94, 6.50). The point below it changes only Y:

$$Y = Y_{\text{box}} + \tfrac{\text{height}}{3} \times 2 = 7.01 \;\Rightarrow\; (9.94,\, 7.01).$$

These are *fractional* feature-map positions — impossible to read directly, which is why interpolation is needed.

Bilinear interpolation

A value P at a fractional point (x, y) lying between the four integer-grid neighbours $Q_{11}, Q_{21}, Q_{12}, Q_{22}$ (at x_1, x_2 and y_1, y_2) is the distance-weighted average

$$P \approx \frac{y_2 - y}{y_2 - y_1}\left(\frac{x_2 - x}{x_2 - x_1}Q_{11} + \frac{x - x_1}{x_2 - x_1}Q_{21}\right) + \frac{y - y_1}{y_2 - y_1}\left(\frac{x_2 - x}{x_2 - x_1}Q_{12} + \frac{x - x_1}{x_2 - x_1}Q_{22}\right).$$

The closer a corner Q_{ij} is to (x, y), the more it counts. This gives a smooth, sub-cell-accurate read of the feature map.

Numeric example: bilinear interpolation at a sample point

Take the sample point $(x, y) = (9.94,\, 6.50)$ sitting in the unit cell with corners at $x_1 = 9.5,\ x_2 = 10.5$ and $y_1 = 6.5,\ y_2 = 7.5$, with feature values

$$Q_{11} = 0.1, \quad Q_{21} = 0.2, \quad Q_{12} = 0.7, \quad Q_{22} = 0.3$$

(at $(x_1, y_1), (x_2, y_1), (x_1, y_2), (x_2, y_2)$ respectively). The weights are x-fractions $\frac{x_2 - x}{1} = 0.56$, $\frac{x - x_1}{1} = 0.44$ and y-fractions $\frac{y_2 - y}{1} = 1.0$, $\frac{y - y_1}{1} = 0.0$. Then

$$P \approx 1.0\,(0.56 \cdot 0.1 + 0.44 \cdot 0.2) + 0.0\,(\dots) = 0.056 + 0.088 = 0.144.$$

Because the point sits exactly on $y = 6.5 = y_1$, the y-weight kills the bottom row entirely and the result is just the horizontal interpolation of the top edge — a useful sanity check.

After interpolating the sample points, each sub-box is max- (or average-)pooled over its 4 points, yielding the fixed-size, *precisely aligned* feature that the mask head needs.

Key idea

Why RoI Align matters. RoI *pooling*'s roundings shift features by up to tens of pixels — invisible to a box classifier but fatal to a mask that must follow object boundaries pixel-by-pixel. RoI *Align*'s quantization-free, interpolated sampling is the single change that makes high-quality instance masks possible, completing the journey from "which class is this image?" (Chapter 1) to "exactly which pixels belong to *this* object?".

Chapter 28

Semantic Segmentation

Chapter 25 placed four tasks on a spectrum from coarse to fine: classification, classification + localisation, detection, and instance segmentation. This chapter develops the dense end of that spectrum. **Semantic segmentation** asks the finest *class-level* question: not "what is in the image?" nor "where are the boxes?", but *"which class does every single pixel belong to?"*. The output is no longer a label or a handful of boxes but a full **label map** the same size as the input.

The chapter first separates the *classical* (non-semantic) notion of segmentation from the *semantic* one, then derives the modern solution as a sequence of engineering fixes — exactly the narrative style of the R-CNN family in Chapter 27. Each design removes the bottleneck left by the previous one: sliding windows are wasteful → go fully convolutional; full-resolution convolutions are too expensive → downsample then upsample; how does one upsample? → unpooling and transposed convolution; and finally we assemble these pieces into the encoder–decoder architectures (U-Net and its variants) that dominate the field. A short final section applies the same machinery to a *regression* version of the dense-prediction problem: monocular depth estimation.

28.1 Classical segmentation: partitioning without meaning

Before the semantic version, recall the *classical* image-processing definition of segmentation, which carries **no semantic meaning at all**. It only asks that the image be cut into homogeneous, non-overlapping pieces.

Classical image segmentation

Let R be the spatial region occupied by an image. Segmentation partitions R into n subregions $R_1, \ldots, R_n$ satisfying:

- $\bigcup_{i=1}^{n} R_i = R$ (the pieces *cover* the whole image);
- each R_i is a **connected set** ($i = 1, \ldots, n$);
- $R_i \cap R_j = \emptyset$ for all $i \neq j$ (the pieces *do not overlap*);
- $Q(R_i) =$ TRUE for every i (each piece is internally *homogeneous*);
- $Q(R_i \cup R_j) =$ FALSE for any *adjacent* R_i, R_j (merging two neighbours would *break* homogeneity).

Here Q is a **logical predicate** defined over the points of a set — for instance $Q(A) =$ TRUE iff all pixels in A share the same intensity level.

Key idea

The last two conditions are the interesting pair. $Q(R_i)$ = TRUE forces each region to be *uniform* (by whatever criterion Q encodes: intensity, colour, texture). $Q(R_i \cup R_j)$ = FALSE for adjacent regions forces the partition to be *maximal* — you cannot glue two touching regions together and stay homogeneous, so the boundaries sit exactly where the image content changes. Together they say: "cut the image along its internal discontinuities, nowhere else."

Numeric example: why both predicates are needed

Take a tiny 1×4 greyscale strip of intensities $[5, 5, 9, 9]$ and let $Q(A)$ = TRUE iff all pixels in A are equal.

- The partition $\{[5, 5], [9, 9]\}$ satisfies *everything*: each piece is uniform (Q = TRUE), and merging the two adjacent pieces gives $[5, 5, 9, 9]$ which is *not* uniform (Q = FALSE). Valid.
- The over-merged partition $\{[5, 5, 9, 9]\}$ fails $Q(R_1)$ = TRUE: the single region is not uniform.
- The over-split partition $\{[5], [5], [9], [9]\}$ satisfies uniformity but fails the adjacency rule: the two touching $[5]$ regions *can* be merged and stay uniform, so $Q(R_i \cup R_j)$ is wrongly TRUE. Not maximal.

Only the first respects both predicates — this is the unique "correct" cut.

Key idea

No semantic meaning. Classical segmentation will happily split a single cat into a "light-fur" region and a "dark-fur" region, or merge a grey cat with a grey rock, because it only knows about *low-level homogeneity* (Q), never about *object categories*. It answers "which pixels look alike?", not "which pixels are *cat*?". Supplying the missing word *cat* is exactly the leap to **semantic** segmentation.

28.2 Semantic segmentation: the task

Semantic segmentation

Label every pixel of the image with a category from a fixed set (e.g. `Sky`, `Grass`, `Trees`, `Cat`, `Cow`). The output is a label map of the same height and width as the input. Crucially, we **do not differentiate instances** — we only care about *pixels*.

Key idea

Class, not instance. This is the single defining limitation, and it echoes the spectrum of Chapter 25. If two cows overlap, semantic segmentation paints *all* their pixels with the one label `cow` and draws *no* boundary between them — the two animals dissolve into one `cow` blob. Separating cow#1 from cow#2 is the job of *instance* segmentation (Mask R-CNN, Chapter 27), which adds the per-instance pixel mask that semantic segmentation lacks. Semantic segmentation = "per-pixel classification"; instance segmentation = "per-pixel classification + which object".

Numeric example: the size of the output

For a modest 100×100 RGB image and $C = 5$ classes, the network must produce a label for each of

$$100 \times 100 = 10{,}000 \text{ pixels.}$$

Internally it predicts a score tensor of shape $C \times H \times W = 5 \times 100 \times 100 = 50{,}000$ numbers (a 5-vector of class scores per pixel), then takes a per-pixel arg max over the 5 classes to collapse this to the final 100×100 label map. Contrast Chapter 25: plain classification emitted *one* label; detection emitted a handful of boxes; here we emit ten-thousand labels — dense prediction is expensive precisely because the output is as large as the input.

28.3 First idea: sliding window (and why it fails)

The most direct way to reuse our classification machinery: **slide a window over the image, and classify the *centre pixel* of each window with a CNN**. Extract a patch around a pixel, push it through the CNN, and whatever class the CNN returns becomes that pixel's label. Do this for every pixel and you have a label map.

This is the pixel-level analogue of the sliding-window detector of Chapter 25, and it inherits the very same disease.

Key idea

Problem: very inefficient — no feature sharing. Adjacent pixels have almost-identical surrounding patches, so the CNN re-extracts essentially the same convolutional features over and over for overlapping windows. We are not *reusing shared features between overlapping patches.* This is the exact redundancy that crippled R-CNN in Chapter 27 (re-running the CNN on ~ 2000 overlapping crops), now multiplied to *one window per pixel.*

Numeric example: the redundancy is enormous

On a 100×100 image, classifying every pixel by its own patch means

$$100 \times 100 = 10{,}000 \text{ independent CNN forward passes per image.}$$

At 5 ms per pass that is 50 s for a single small image — and almost all of that work is wasted, because the patch around pixel $(50, 50)$ and the patch around its neighbour $(50, 51)$ overlap in all but one column, so their early convolutional features are nearly identical. The fix, exactly as for detection, is to compute the shared convolutional features *once* for the whole image.

28.4 Second idea: fully convolutional network

The cure for the redundancy is to drop the per-pixel cropping entirely and make the network **fully convolutional**: a stack of convolution layers that turns the whole image into the whole label map in one pass.

Fully convolutional network (FCN)

Feed the input image ($3 \times H \times W$) through a stack of convolutions that preserve spatial size, ending in a layer with C output channels. This produces a **score tensor** of shape $C \times H \times W$: at every pixel, a C-vector of class scores. A per-pixel arg max over the C channels yields the final **prediction map** of shape $H \times W$.

Key idea

Why this beats the sliding window. Because convolution is applied to the whole image at once, overlapping receptive fields automatically *share* their computation — the feature at each location is computed exactly once. One forward pass labels *all* pixels, instead of one pass per pixel. (This is the segmentation counterpart of Fast R-CNN's "run the convnet once on the whole image" insight from Chapter 27.)

Key idea

New problem: full-resolution convolutions are very expensive. Keeping the feature maps at the original $H \times W$ through *every* layer means every convolution operates on $H \times W$ spatial positions with D channels. The cost and memory of a conv layer scale with $H \times W \times D$, so a deep stack at full resolution is prohibitive. We need to shrink the spatial size inside the network — but the output must still come back out at full resolution.

Numeric example: cost of staying at full resolution

One 3×3 convolution mapping D_{in} channels to D_{out} channels over an $H \times W$ map costs about

$$H \times W \times D_{\text{in}} \times D_{\text{out}} \times 3 \times 3 \text{ multiply–adds.}$$

For $H = W = 512$, $D_{\text{in}} = D_{\text{out}} = 64$:

$$512 \times 512 \times 64 \times 64 \times 9 \approx 9.7 \times 10^{10} \approx 97 \text{ billion operations — for a } \textit{single} \text{ layer.}$$

Halving the resolution to 256×256 divides the spatial term by 4, cutting this one layer to ~ 24 billion. With a deep stack of such layers the saving compounds, which is exactly why we downsample inside the network.

28.5 The encoder–decoder shape: downsample, then upsample

The resolution dilemma is resolved by designing the network as a bunch of convolutional layers with **downsampling** and **upsampling** *inside* the network.

Downsample–upsample design

First half (encoder / downsampling): progressively *reduce* spatial resolution (e.g. $H \times W \to H/2 \times W/2 \to H/4 \times W/4$) while typically *increasing* channel depth. Expensive convolutions thus run on small feature maps. This compresses the image into a low-resolution, semantically rich representation.

Second half (decoder / upsampling): progressively *increase* the resolution back up to the original $H \times W$, so the final prediction map has the same size as the input.

Key idea

Where the cost goes. The heaviest convolutions now act at low resolution (e.g. $H/4 \times W/4$), where each layer touches $16\times$ fewer spatial positions than at full size. The network still *outputs* a full $H \times W$ map because the decoder restores the resolution. This leaves one question, the engine of the next two sections: **how do we upsample?**

28.5.1 Downsampling: the tools we already have

Downsampling is the familiar part — we have two standard mechanisms.

Two ways to downsample

Pooling (max or average): slide a small window (e.g. 2×2) and replace it by a single number — its maximum (*max pooling*) or its mean (*average pooling*). **No parameters** are learned here. Pooling buys some *local invariance*, at the price of *discarding information*.

Strided convolution: a convolution that hops by a stride > 1, so its output map is smaller than its input. Unlike pooling this *does* have learnable weights.

Numeric example: max vs. average pooling on a 2×2 block

Take the top-left 2×2 block of the slide's grid,

$$\begin{bmatrix} 2 & 1 \\ 5 & 0 \end{bmatrix}.$$

Max pooling returns $\max(2, 1, 5, 0) = 5$. *Average* pooling returns $\frac{2+1+5+0}{4} = \frac{8}{4} = 2.0$. Max pooling keeps the strongest activation (good for "did this feature fire *anywhere* in the block?"), while average pooling smooths. Either way a 2×2 block becomes one number, so an $H \times W$ map shrinks to $\frac{H}{2} \times \frac{W}{2}$, and *three of the four original values are gone* — the information loss the decoder must somehow undo.

28.6 Upsampling I: unpooling

The simplest upsamplers are *fixed* (no learning): they just spread small maps into larger ones by a rule.

Three fixed unpooling rules

Mapping a 2×2 input to a 4×4 output:

- **Nearest neighbour:** copy each input value into its whole 2×2 output cell (replicate).
- **"Bed of nails":** place each input value in *one* fixed position (say the top-left) of its 2×2 cell, and fill the other three with 0.
- **Max unpooling:** like bed-of-nails, but put each value back at the *exact position* from which the corresponding max-pool earlier took its maximum. This requires **remembering the argmax positions** from the paired pooling layer.

Numeric example: the three rules on the same input

Let the 2×2 input be $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.

Nearest neighbour replicates each entry into a 2×2 block:

$$\begin{bmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 3 & 4 & 4 \\ 3 & 3 & 4 & 4 \end{bmatrix}.$$

Bed of nails (value at top-left of each cell, zeros elsewhere):

$$\begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Both are pure rules with nothing learned.

Key idea

Why max unpooling is the clever one. Max pooling threw away *where* the maximum was; if we record that location, max unpooling can drop the value back *exactly* where it came from, restoring spatial precision at object boundaries. This is why the architecture is drawn with **corresponding pairs of downsampling and upsampling layers**: each unpooling layer is wired to its matching pooling layer to reuse the stored argmax positions. It is the same "don't lose the location" concern that motivated RoI *Align* over RoI pooling in Chapter 27.

Numeric example: max pooling then max unpooling, end to end

Max-pool the 4×4 map (with 2×2 windows, stride 2)

$$\begin{bmatrix} 1 & 2 & 6 & 3 \\ 3 & 5 & 2 & 1 \\ 1 & 2 & 2 & 1 \\ 7 & 3 & 4 & 8 \end{bmatrix}.$$

Per quadrant the maxima are: top-left $\max(1,2,3,5) = 5$ (at row 2, col 2); top-right $\max(6,3,2,1) = 6$ (row 1, col 3); bottom-left $\max(1,2,7,3) = 7$ (row 4, col 1); bottom-right $\max(2,1,4,8) = 8$ (row 4, col 4). So pooling gives $\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ and *records* those four positions. After the rest of the network returns a 2×2 map — say $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ — max unpooling writes each value back to its stored slot:

$$\begin{bmatrix} 0 & 0 & 2 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 4 \end{bmatrix}.$$

The non-zero entries land precisely where the original maxima were — spatial detail recovered for free.

28.7 Upsampling II: transposed convolution (learnable)

Unpooling rules are fixed. To *learn* how to upsample, we use a **transposed convolution** (also called fractionally-strided or, loosely, "deconvolution"). The slides build it from the relationship between ordinary convolution and its reverse.

28.7.1 The set-up: ordinary convolution as the forward map

The forward (downsampling) convolution

The slides start from a 4×4 image convolved with a 3×3 filter, producing a 2×2 output. Each output number is the dot product of the filter with one 3×3 patch of the input. Going *forward*, many input pixels collapse to one output pixel. Transposed convolution does the **opposite**: it expands *one* input pixel into a whole output patch.

28.7.2 The mechanism: one output map per input pixel, then overlap-add

Transposed convolution

For *each* pixel value v of the small input, multiply the *whole filter* by v and **paste** the resulting scaled-filter patch into the large output, positioned according to that pixel's location (stepping by the stride). Different input pixels produce patches that **overlap**; wherever they overlap, **sum** the contributions. The accumulated map is the upsampled output.

Key idea

"One output map for each pixel." Each input pixel v generates its own copy of the filter scaled by v (so input value 55 with filter entry 2 contributes 110, etc.). These per-pixel maps are laid down at shifted positions and *added* where they overlap. Because the filter weights are **learnable**, the network *learns how to upsample* rather than being stuck with a fixed rule — the slides stress: "values are different! transpose convolution kernels are learnable (training)."

Numeric example: a 1-D transposed convolution worked out

Take input vector [2, 3] and a learnable filter [1, 2, 1], output stride 1 (so consecutive input pixels are placed one step apart).
Pixel 2 generates $2 \times [1, 2, 1] = [2, 4, 2]$, placed at offset 0:

$$[\,2,\ 4,\ 2,\ 0\,].$$

Pixel 3 generates $3 \times [1, 2, 1] = [3, 6, 3]$, placed at offset 1:

$$[\,0,\ 3,\ 6,\ 3\,].$$

Overlap-add the two:

$$[\,2,\ 4+3,\ 2+6,\ 3\,] = [\,2,\ 7,\ 8,\ 3\,].$$

Two input values became four output values, with the overlap region $(7, 8)$ mixing contributions from both — the 1-D analogue of the slides' 2-D overlap-and-sum, and the values change as the filter is trained.

28.8 A basic architecture: the U-Net

We can now assemble the pieces (encoder downsampling, decoder upsampling) into a concrete, hugely influential network.

U-Net

U-Net (Ronneberger, Fischer & Brox, 2015) is an **encoder–decoder** network originally built for **biomedical image segmentation**. The left arm (*contracting path*) repeatedly applies 3×3 convolutions + ReLU and 2×2 max pooling, halving resolution and doubling channels. The right arm (*expanding path*) repeatedly applies 2×2 up-convolutions (transposed convolution) to restore resolution, and a final 1×1 convolution maps the feature channels to the C class scores. Drawn out, the contract-then-expand shape forms the letter "U".

Key idea

The key ingredient: skip connections ("copy and crop"). At each resolution level, U-Net *copies* the encoder's feature map and concatenates it onto the matching decoder feature map. The decoder thus sees *both* the coarse, semantically rich features brought up from below *and* the fine, high-resolution detail carried across from the encoder. Downsampling discards spatial precision (Section 6); the skip connections hand that precision directly to the decoder, which is what lets U-Net produce crisp boundaries. This is the same "route the lost detail around the bottleneck" idea as max-unpooling's stored positions, generalised to whole feature maps. U-Net won the ISBI 2015 cell-tracking and dental-radiography caries challenges.

Numeric example: tracking the resolution down and back up

The original U-Net takes a 572×572 tile. The contracting path roughly halves the spatial size at each of four pooling steps:

$$572 \to 284 \to 140 \to 68 \to 32 \quad \text{(approx., before each pool)},$$

while channels grow $1 \to 64 \to 128 \to 256 \to 512 \to 1024$. The expanding path then doubles the size back up four times via up-convolutions ($32 \to 64 \to 104 \to 200 \to 392$, approx.), with channels shrinking $1024 \to 512 \to 256 \to 128 \to 64$, ending in a 1×1 conv to $C = 2$ classes and a 388×388 output map. Resolution is *lost* on the way down (cheap, deep convolutions) and *rebuilt* on the way up, with skip connections restoring the detail that pooling discarded.

28.9 U-Net variants

Several widely used architectures keep U-Net's encoder–decoder skeleton and change one ingredient.

Three U-Net variants

LinkNet: an encoder–decoder built on a **ResNet** backbone (e.g. ResNet-34). Its skip connections use **summation** rather than concatenation, and each *decoder block* is a small conv1 $\times$ 1 $\to$ transposed conv $\to$ conv1 $\times$ 1 unit (with batch-norm). Summation keeps the decoder lightweight (no growth in channel count from concatenation), making LinkNet efficient.

PSPNet (Pyramid Scene Parsing Network): after a CNN produces the feature map, a **pyramid pooling module** pools it at *several different region sizes* (coarse to fine) in parallel, processes each with a convolution, *upsamples* them all back to a common size, and *concatenates* them. This fuses **local and global context**, which is why PSPNet outperformed the prior state of the art on segmentation in 2017.

FPN (Feature Pyramid Network): combines a **bottom-up** pathway (a normal CNN whose resolution *decreases* and whose *semantic value increases* as you go up) with a **top-down** pathway that propagates the strong high-level semantics back down to the high-resolution layers, predicting at *multiple* scales.

Key idea

The unifying theme. Every variant is fighting the same tension introduced in Section 6: deep features are *semantically strong but spatially coarse*, shallow features are *spatially precise but semantically weak*. Concatenation skips (U-Net), summation skips (LinkNet), multi-scale pooling (PSPNet) and the top-down pathway (FPN) are four different ways to **fuse** coarse semantics with fine spatial detail. They differ in *how* they merge, not in *why*.

Numeric example: why multi-scale context helps (PSPNet)

A single car pixel in a street scene is ambiguous from a tiny neighbourhood — a patch of dark pixels could be car, road, or shadow. Pooling the feature map at a *coarse* level (say a single 1×1 summary of the whole map) injects the global hint "this is a street scene"; pooling at a *finer* level (e.g. a 3×3 grid) preserves "this blob is car-shaped here". PSPNet pools at several such levels at once — e.g. grids of 1×1, 2×2, 3×3, 6×6 — upsamples each back to the feature-map size, and concatenates. The per-pixel classifier then sees its local evidence *and* nested summaries of ever-larger surroundings, resolving the ambiguity.

28.10 A regression cousin: monocular depth estimation

The same dense-prediction machinery solves a problem that is *not* classification at all. Where semantic segmentation outputs a *class* per pixel, **monocular depth estimation** outputs a *real number* per pixel

— it is the **regression** version of dense prediction, just as the localisation head of Chapter 27 was the regression version of a classification head.

Monocular depth estimation

Given a **single** (monocular) RGB image, estimate the **depth value** — the distance relative to the camera — of *each* pixel. The output is a **depth map** of shape $H \times W$ of continuous values, rather than a categorical label map. "Monocular" is what makes it hard: with one image there is no stereo disparity to triangulate from, so depth must be inferred from monocular cues (perspective, texture gradients, known object sizes, occlusion).

Key idea

Two families of methods, and the data bottleneck. State-of-the-art approaches either *(i)* design one powerful network that **directly regresses** the whole depth map, or *(ii)* **split the input into bins or windows** to cut computational cost. The recurring practical obstacle is a **lack of ground-truth datasets for supervised training**: dense per-pixel depth labels require special sensors (LiDAR, structured light) and are far scarcer and noisier than the segmentation masks a human can simply paint — which pushes the field toward synthetic data and self-/temporally-supervised training.

Numeric example: classification vs. regression output, side by side

For a 100×100 image:

- **Semantic segmentation** predicts a $C \times 100 \times 100$ score tensor, then arg max over the C classes $\Rightarrow$ a 100×100 map of *discrete labels*, trained with a per-pixel *cross-entropy* loss.
- **Depth estimation** predicts a single $1 \times 100 \times 100$ map of *continuous* distances (e.g. metres), trained with a per-pixel *regression* loss such as L2, $\sum_p (\hat{d}_p - d_p)^2$ over the 10,000 pixels.

The architecture (encoder–decoder, skip connections) is essentially the same; only the final layer and the loss change — C-way softmax for segmentation versus a single regressed value for depth.

An example system: M4Depth. As a concrete instance, M4Depth (Fonder, Ernst & Van Droogenbroeck, 2022) tackles *temporal* monocular depth in unstructured environments. It is an encoder–decoder operating on a *video sequence*: an encoder extracts multi-scale features $f_{\text{enc}}^{1,2,3}$ from each frame, and a decoder of stacked "parallax refiner" stages, fed with the camera motion $(\mathbf{R}_t, \mathbf{t}_t)$ between frames, infers **parallax** (apparent pixel shift due to motion) and converts it $\rho \to d$ into a depth map per frame. It illustrates the section's two points at once: it reuses the segmentation-style encoder–decoder skeleton, and it sidesteps the ground-truth-data bottleneck by exploiting *motion across frames* as an extra cue rather than relying solely on static supervised labels.

Closing remark

This chapter pushed the Chapter 25 spectrum to its dense-prediction extreme. Classical segmentation cut images by low-level homogeneity *without* meaning; semantic segmentation attaches a *class* to every pixel; and the machinery to do it efficiently — fully convolutional networks, the encoder–decoder down-sample/upsample shape, learnable transposed-convolution upsampling, and skip connections — recurs, with only the output head swapped, in the regression cousin of depth estimation. The throughline from Chapter 27 holds: each architectural step exists to remove the bottleneck (redundant computation, then full-resolution cost, then lost spatial detail) left by the one before it.